# “College ERP Django”

**A**

**Dissertation Work**

***Submitted in partial fulfillment of the requirement for the award of Degree of***

**Master of Computer Applications**

**Submitted to**


**Department of Computer Science**
**CIMAGE Center of Digital Technology & Entrepreneurship**
**Patliputra Branch, Block-2**

**Submitted By**
**Aayush Sahay**
Enrollment No:24326711019

MCA-IV Semester

**Under Guidance of**
**Dr. Amit Kumar Shukla**



**Department of Computer Science**
**CIMAGE Center of Digital Technology & Entrepreneurship**

**Session 2024-2026**

**DEPARTMENT OF COMPUTER SCIENCE**
**CIMAGE Center of Digital Technology & Entrepreneurship**

---

**Date:……**

# CERTIFICATE

This is to certify that the work embodies in this dissertation entitled, **",College ERP django"** being submitted by **Aayush Sahay 24326711019** in partial fulfillment of the requirement for the award of "**Master of Computer Applications"** to **CIMAGE Center of Digital Technology & Entrepreneurship** during the academic year 2024-2026 is a record of bonafide piece of work, carried out by him under our/my supervision and guidance in the **"Department of Computer Science", CIMAGE Center of Digital Technology & Entrepreneurship.**

**Dr. Amit Kumar Shukla**
Deptt. Of Computer Science

DEPARTMENT OF COMPUTER SCIENCE

CIMAGE Center of Digital Technology & Entrepreneurship

Date:----------
---

# APPROVAL CERTIFICATE

The dissertation entitled **"College ERP django "** being submitted by **Aayush Sahay 24326711019** has been examined by us and is hereby approved for the award of degree "**Master of Computer Applications",** for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, the opinion expressed or conclusion drawn therein, but approve the dissertation only for the purpose for which it has been submitted.

(Internal Examiner) (External Examiner)

Date:- Date:-

**Date:----------**

**---**

# DECLARATION

I **Aayush Sahay** hereby declare that the work, which is being presented in the dissertation entitled **"College ERP Django"** partial fulfillment of the requirements for the award of degree of "**Master of Computer Applications"** submitted in the Department of **Computer Science** is an authentic record of my own work carried under the guidance of. I have not submitted the matter embodied in this report for the award of any other degree.

I also declare that' "A check for Plagiarism has been carried out on the thesis/project report/ dissertation and is found within the acceptable limit and report of which is enclosed herewith."

**Aayush Sahay**
Enrollment No 24326711019
**MCA** IV Sem

## ACKNOWLEDGEMENT

*"A journey is easier when you travel together. Interdependence is certainly more valuable than independence."*

I would like to thanks, **Dr. Amit Kumar Shukla**, for providing guidance and insight into my research work. I also thank his for all advice he has given me in the past year, and for always having time for me, whenever I needed.

I give special thanks to **Raju Upadhyay Sir,** and **Ravi Kumar Soni Sir, Murali Manohar Sir, Anjesh Sir** and **Niraj Kumar Singh Sir** for always being willing to help find solutions to any problems I had with my work.

***"The completion of any project depends upon the cooperation, coordination, and combined efforts of several resources of knowledge, inspiration, and energy".***

I also extend my deepest gratitude to **Director**, CIMAGE GROUP OF INSTITUITIONS, PATNA, **Dr. Neeraj Agrawal Sir, Dean** CIMAGE GROUP OF INSTITUITIONS, PATNA, **Dr. Neeraj Kumar Sir** and **Centre Head Mrs. Megha Agrawal Ma'am** for providing all the necessary facilities and true encouraging environment to bring out the best in my endeavours.

I express my gratitude and thanks to all the staff members of Computer Science departments for their sincere cooperation in furnishing relevant information to complete this Project well in time successfully.

I extend a special word to my friends, who have been a constant source of inspiration throughoutmy project work.

Lastly but not least I must express my cordial thank to my parent and family members who gave me the moral support without that it is impossible to complete my project work. With this note, I thank everyone for the support.

**(Aayush Sahay)**
Enrollment No : 24326711019
MCA IV Sem

# ABSTRACT

The College ERP System is a comprehensive web-based enterprise application developed using the Django framework to automate and streamline the academic, administrative, and management processes of a higher education institution. The system is designed to provide a centralized platform for managing students, faculty members, courses, academic sessions, attendance, examinations, admissions, and administrative operations. Built following a modular architecture, the application enables secure role-based access for administrators, faculty, and students, ensuring efficient information management and real-time data accessibility. The backend is powered by PostgreSQL, providing reliable and scalable data storage, while Django's robust ORM and security features facilitate maintainable development and secure transaction processing. The system aims to reduce manual paperwork, improve operational efficiency, enhance data accuracy, and support informed decision-making through a unified digital platform.

To ensure scalability, reliability, and ease of deployment, the College ERP System is containerized using Docker and hosted on Amazon Web Services (AWS) Elastic Compute Cloud (EC2). Docker enables consistent application deployment across environments by packaging the application and its dependencies into isolated containers, while PostgreSQL operates as the primary relational database management system. The AWS cloud infrastructure provides high availability, remote accessibility, and flexible resource management, making the system suitable for institutional-level deployment. Through the integration of Django, PostgreSQL, Docker, and AWS EC2, the proposed ERP solution delivers a secure, scalable, and cost-effective platform capable of supporting the evolving digital transformation needs of educational institutions while improving collaboration among students, faculty, and administrative staff.

# TOPIC OF THE PROJECT



| | |
|---|---|
| **Developed By :** | **AAYUSH SAHAY** |
| **Enrollement No :** | **24326711019** |
| **Session :** | **2024-2026** |
| **Institution :** | **Cimage Centre of Digital Technology and Entrepreneurship, PATNA** |
| **College Code :** | **711** |
| **Under Guidance OF:** | **Dr. Amit Kumar Shukla (HOD-IT)** |


**Submitted To ARYABHATTA KNOWLEDGE UNIVERSITY , PATNA**
**in Partial fulfillement of the Requirements for the Award of the Degree**

## Masters in Computer Applications (MCA)

# Contents

# Chapter : 1

# Introduction

# College ERP

# Introduction To The Project : College ERP Django and AWS

Streamlining a College , Students, faculty, administrative staff, Faculties
and the top Management.

This project is a web-based College ERP System built using Django, containerized with Docker, version-controlled via GitHub, and deployed on AWS using free-tier eligible services for demonstration purposes.

The system is designed to manage and streamline operations across multiple roles including staff, faculty, administrative personnel, and top-level management. It provides a centralized platform for handling academic, administrative, and operational workflows efficiently.

Key objectives:

- Unified management of users, roles, and permissions
- Digitization of academic and administrative processes
- Scalable and modular architecture for future feature expansion
- Cloud deployment for accessibility and real-time usage

The architecture ensures flexibility to add or disable modules as requirements evolve, making it adaptable for future enhancements while maintaining simplicity for demonstration and initial deployment.

# Chapter : 2

# Objectives of the Project

# College ERP

Objectives of : College ERP

This project is a simplified, modular College ERP System designed with a strong DevOps-oriented workflow. It is built using Django, containerized with Docker, version-controlled via GitHub, and deployed on AWS (free-tier eligible services) for demonstration.

The system focuses on managing core institutional roles—staff, faculty, administrative personnel, and top-level management—through a clean, structured architecture that is easy to develop, maintain, and extend.

The design follows a modular app-based approach in Django, allowing:

- Easy addition or removal of features (apps) based on requirements
- Clear separation of concerns for maintainability
- Smooth integration of new modules without affecting existing ones

From a DevOps perspective, the project enables:

- Continuous code integration and version control via GitHub
- Containerized development and deployment using Docker
- Simplified deployment pipeline on AWS infrastructure
- Consistent environments across development and production

The development process aligns with Agile practices:

- Iterative feature development
- Continuous testing and validation
- Incremental updates and deployments

Overall, the system emphasizes simplicity, structured development, ease of management, and seamless integration—serving both as a functional ERP prototype and a complete DevOps + Software Engineering demonstration in a single, cohesive setup.

# Chapter : 3

# Features of the Project

# College ERP

## Features of the College ERP

i) **Security-Oriented Architecture**

- Address common security limitations of traditional PHP-based hosting
- Leverage Django's built-in security features (authentication, CSRF protection, ORM safety)

ii) **Developer-Friendly Technology Stack**

- Use Python for faster development, readability, and maintainability
- Enable efficient backend logic and rapid feature implementation

iii) **Flexible UI Integration**

- Support frontend development using HTML, CSS, JavaScript, or React
- Allow seamless integration of modern, responsive user interfaces

iv) **Modular Django Framework Design**

- Build system using independent Django apps
- Add, remove, disable, or create apps as per evolving requirements
- Maintain clean, structured, and scalable codebase

v) **Cloud Deployment (AWS Free Tier)**

- Deploy application using AWS free-tier services for demonstration
- Provide temporary public access via hosted URLs
- Ensure basic scalability and availability

vi) **Containerization with Docker**

- Standardize development and deployment environments
- Simplify setup, testing, and portability across systems

vii) **Version Control with GitHub**

- Maintain source code with proper version tracking
- Enable collaboration, code updates, and rollback capability

viii) **DevOps-Driven Workflow**

- Streamline coding → version control → deployment pipeline
- Ensure continuous integration and easy deployment cycles

ix) **Agile Development & Testing**

- Follow iterative development approach
- Support continuous testing and incremental feature updates

x) **Structured & Maintainable System Design**

- Keep the project simple, organized, and easy to manage
- Ensure future extensibility and smooth integration of new features

# Chapter : 4

# Project Category

# College ERP

## Category of the Project

The proposed College ERP (Enterprise Resource Planning) system is designed as a comprehensive digital solution to manage and automate the academic and administrative processes of an educational institution. The project is developed by integrating core principles from multiple domains of software engineering, ensuring a robust, scalable, secure, and user-friendly system.

At the foundation, the system follows the **principles of software analysis and design**, where requirements are carefully gathered, analyzed, and structured into functional and non-functional components. This ensures clarity in system objectives, proper module division (such as student management, faculty management, attendance, examinations, etc.), and a well-defined architecture that supports future scalability.

The development process adheres to **Agile methodology**, a key principle of software engineering. Agile enables iterative development, continuous feedback, and adaptive planning. The ERP system is built in incremental modules, allowing testing, validation, and improvements at each stage. This approach enhances flexibility and reduces development risks.

Incorporating **software design and management principles**, the system emphasizes modularity, reusability, and maintainability. Proper design patterns, layered architecture, and separation of concerns are applied to ensure that each component functions independently yet cohesively within the system. Project management practices are followed to track progress, manage resources, and ensure timely delivery.

Quality assurance is maintained using **software testing principles**, including unit testing, integration testing, and system testing. These practices ensure that each module works correctly, interactions between modules are seamless, and the overall system meets performance and reliability standards.

The system also integrates **DevOps and cloud deployment principles**, enabling continuous integration and continuous deployment (CI/CD). This ensures faster delivery cycles, automated testing pipelines, and efficient deployment on cloud platforms, improving accessibility and scalability.

From a technical infrastructure perspective, **networking principles** are applied to ensure smooth communication between system components, especially in multi-user environments. Proper handling of client-server architecture, request-response cycles, and data transmission is considered.

Security is a critical aspect, addressed through **cybersecurity and risk prevention principles**. The system is designed with a proactive approach to minimize vulnerabilities, including secure authentication, data encryption, access control, and protection against common threats such as SQL injection and unauthorized access. The objective is to eliminate potential flaws and ensure data integrity and confidentiality.

User experience is enhanced by applying **UI designing principles**, focusing on intuitive layouts, responsive design, and ease of navigation. The interface is structured to be user-friendly for students, faculty, and administrators, ensuring minimal learning curve and efficient usage.

Efficient data handling is achieved through **database management principles**, ensuring proper schema design, normalization, indexing, and optimized queries. This results in reliable data storage, quick retrieval, and consistency across the system.

To ensure system reliability, **troubleshooting principles** are implemented, enabling quick identification and resolution of issues. Logging mechanisms, error handling, and debugging practices are integrated into the system.

Finally, the project emphasizes **maintenance and training principles**, ensuring long-term usability. Proper documentation, user manuals, and training support are provided so that end-users can effectively operate the system. Regular updates and maintenance strategies are planned to keep the system aligned with evolving requirements.

Overall, the College ERP system represents a holistic application of multidisciplinary principles, combining software engineering, security, cloud computing, and user-centric design to deliver an efficient and future-ready solution.

**Quick Overview:**

i. principles of software analysis and design
ii. principles of software engineering : Agile methodology
iii. principles of software design and management.
iv. principles of software testing
v. principles of Devops and cloud Deployment
vi. principles of Networking
vii. principles of cybersecurity and risk prevention [promise of keeping in mind to terminate the possibilities of flaws / loose ends]
viii. principles of UI Designing
ix. principles of database usage
x. principles of trouble shooting &
xi. principles of maintainence and training

# Chapter : 5

# Scope of the Project

# College ERP

## Scope of the Project: College ERP

i. Use of **Python** enables future integration with AI and ML modules (predictive analytics, student performance analysis, automation).

ii. Python's developer-friendly nature supports faster feature expansion and easier maintenance.

iii. Frontend can evolve from standard HTML, CSS, JS to modern frameworks like **React** for improved UI/UX and dynamic interfaces.

iv. **Django** allows modular expansion (add/remove apps, scalability, microservices adaptation).

v. Deployment scalability using **Amazon Web Services** (AWS Free Tier initially, later full cloud scaling).

vi. Use of **PostgreSQL** ensures robust, scalable, and cloud-compatible database management.

vii. Linux-based cloud environments enhance performance, security, and cost-efficiency.

viii. Containerization using **Docker** ensures consistent environments across development and production.

ix. Version control and collaboration via **GitHub** enables team scalability and CI/CD integration.

x. Future integration with automation tools, analytics dashboards, and mobile app support.

xi. The use of the Django framework provides a modular architecture, making it easy to add, remove, or upgrade individual applications within the ERP system. This flexibility supports future scalability and enables seamless integration of new modules such as analytics dashboards, communication systems, or mobile APIs.

xii. For deployment, the system leverages Amazon Web Services (AWS), starting with the free tier for initial hosting and demonstration. In the future, it can be scaled to a full cloud-based infrastructure, supporting high availability, load balancing, and global access. The use of PostgreSQL as the database ensures reliability, performance, and compatibility with cloud-based environments, particularly in Linux-based systems which are widely used in production.

xiii. Containerization using Docker further enhances the future scope by enabling consistent and portable environments across development, testing, and production stages. This simplifies deployment and reduces configuration-related issues. Additionally, the use of GitHub for version control supports collaborative development, continuous integration, and efficient project management.

xiv. Overall, the project is designed with a future-ready approach, allowing integration of emerging technologies, scalability in cloud environments, and continuous improvement in functionality, security, and user experience.

# Chapter : 6

# Hardware and Software Requirements For the Project

# College ERP

# Hardware and Software Requirements : College ERP

The development and deployment of the College ERP system are designed to be cost-effective and scalable by utilizing cloud-based infrastructure and open-source technologies.

On the hardware side, the system leverages a cloud-based virtual machine instead of traditional physical servers. A free-tier instance from **Amazon Web Services** running **Ubuntu** is used as the primary environment for hosting and testing the application. This ensures accessibility, flexibility, and real-time deployment capabilities. For storage management, Elastic Block Storage (EBS) is utilized to handle data volumes efficiently, allowing scalable and reliable data storage. The system also supports containerized environments using **Docker**, which ensures consistency across development and deployment stages while reducing dependency-related issues.

On the software side, the development process relies entirely on free and open-source tools. The primary code editor used is **Visual Studio Code**, which provides a lightweight yet powerful development environment with extensive plugin support. For UI/UX prototyping, **Penpot** is used, enabling collaborative and open-source interface design.

The backend of the system is developed using **Python** along with the **Django**, ensuring rapid development, scalability, and clean architecture. For database management, **PostgreSQL** is used due to its robustness, performance, and compatibility with cloud environments.

Containerization is handled using Docker, and optional container orchestration services can be integrated in the future for scaling purposes. Version control and collaborative development are managed using **GitHub**, enabling efficient tracking of changes, team collaboration, and continuous integration workflows.

Overall, the selected hardware and software stack ensures a reliable, scalable, and modern development ecosystem while maintaining cost efficiency and flexibility for future expansion.

# Chapter : 7

# Problem Definition & Requirements Gathering

# College ERP

# Problem Definition And Requirements Gathering : College ERP

## Problem Definition

Managing academic and administrative operations in colleges has become increasingly complex, especially for institutions handling a large number of students and resources. Traditional or semi-digital systems often lead to fragmented data, inefficient workflows, lack of real-time updates, and difficulty in maintaining consistency across departments. This creates challenges in managing student records, inventory, events, and institutional information effectively.

For larger institutions with high student volumes, the need for a centralized and scalable system becomes critical. Such institutions, particularly those capable of investing in cloud infrastructure, require innovative solutions that can handle large-scale operations efficiently. The absence of an integrated platform often results in poor data management, duplication of records, and delays in accessing important information.

Another major issue is the lack of proper systems to manage multiple domains such as student data, inventory tracking, event management, and the college's public-facing website. These components are often handled separately, leading to inconsistencies and increased administrative effort. Additionally, there is a strong need for continuously updated content, as outdated information on portals or websites can affect communication, decision-making, and institutional credibility.

Current systems also fall short in terms of deployment and monitoring. Many solutions are not easily deployable on modern cloud environments, nor do they provide efficient tracking and analytical capabilities. Institutions require systems that are easy to deploy, monitor, and analyze, enabling administrators to gain insights from data and make informed decisions.

To address these challenges, there is a need for a robust ERP solution that ensures efficient data management and supports analytical capabilities. Proper database design principles, including normalization, are essential to maintain data integrity, reduce redundancy, and improve performance. The proposed solution aims to implement such a system using the **Django** framework, providing a scalable, modular, and efficient platform to manage all core institutional operations in an integrated manner.

## Problem Recognition

In the current academic environment, many colleges continue to rely on disconnected systems or manual processes to manage their operations. This fragmented approach makes it difficult to handle growing data volumes, especially in institutions with a large number of students. The lack of a unified platform results in inefficiencies in managing student records, inventory, events, and institutional information, ultimately affecting productivity and decision-making.

A key problem identified is the absence of centralized data management. Different departments often maintain separate records, leading to redundancy, inconsistency, and increased chances of errors. Without proper integration, retrieving and updating information becomes time-consuming and unreliable. This issue becomes more critical for larger institutions, where the scale of operations demands a more structured and scalable solution.

Another important concern is the lack of real-time updates and content management. Institutional websites and internal systems often display outdated information, which can lead to miscommunication among students, faculty, and administration. There is a clear need for a system that ensures continuous content updates and synchronization across all modules.

The current systems also lack effective tools for tracking, monitoring, and analyzing data. Institutions are unable to fully utilize their data for insights such as student performance trends, resource utilization, or event effectiveness. This highlights the need for analytical capabilities within the system to support data-driven decision-making.

Deployment and maintenance challenges further complicate the situation. Many existing solutions are not designed for easy deployment on cloud platforms, making scalability and accessibility difficult. Institutions require systems that are easy to deploy, manage, and monitor, particularly those capable of leveraging cloud infrastructure for better performance and reach.

Additionally, poor database design practices in existing systems often result in data redundancy and inefficiency. The absence of normalization and structured schema design leads to performance issues and difficulties in maintaining data integrity.

Recognizing these challenges, it becomes evident that there is a need for an integrated, scalable, and efficient ERP solution. A system built using modern frameworks like **Django** can address these gaps by providing modular architecture, centralized data management, real-time updates, and analytical capabilities, thereby improving overall institutional efficiency.

## Preliminary Investigations

- Identified reliance on manual and semi-digital systems across departments leading to inefficiencies.
- Lack of a centralized platform for managing student data, faculty records, inventory, and events.
- Data redundancy and inconsistency observed due to unstructured and non-normalized databases.
- Difficulty in handling large-scale data for institutions with high student intake.
- Existing systems fail to provide real-time updates, resulting in outdated information on portals and websites.
- Absence of integration between academic, administrative, and public-facing modules.
- Limited capability for tracking, monitoring, and analyzing institutional data.
- Inefficient inventory and resource management due to lack of automation.
- Event management processes are not streamlined or properly recorded.
- Deployment challenges observed in traditional systems; lack of cloud readiness.
- Institutions with access to cloud infrastructure require scalable and innovative solutions.
- Need for systems that are easy to deploy, manage, and maintain in real-time environments.
- Security concerns due to weak access control and lack of structured data handling.
- Requirement for a user-friendly interface for students, faculty, and administrators.
- Need for proper database design using normalization to ensure data integrity and efficiency.
- Identified importance of analytical features for decision-making and performance evaluation.
- Proposed solution involves developing a modular ERP system using **Django**.
- System will support integration of multiple modules such as data management, inventory, events, and public site.
- Future readiness ensured through scalability, cloud deployment capability, and analytical support.

This preliminary investigation confirms the need for a comprehensive, integrated, and scalable College ERP system to overcome existing operational limitations.

# Requirements Specifications:-

### 1. Point-wise Requirement Specification (Highly Customizable Cloud-Based Software)

- System must be cloud-based for accessibility, scalability, and remote usage.
- Highly customizable modules to suit different institutional needs (students, faculty, inventory, events).
- Centralized database for unified data management across departments.
- Real-time data updates and synchronization across all modules.
- Secure authentication and role-based access control.
- Easy deployment, monitoring, and maintenance on cloud platforms.
- Analytical capabilities for reports, insights, and decision-making.
- User-friendly interface for students, faculty, and administrators.
- Integration capability with future technologies (AI/ML, mobile apps).
- Reliable performance even with large-scale data (high student volume).
- Proper database design with normalization to ensure data integrity and efficiency.

### 2. Current Systems

- Use of spreadsheets and online tools (e.g., Excel, shared sheets) for managing records.
- Data stored in isolated formats with no central integration.
- High dependency on network reliability for accessing shared sheets.
- Increased chances of data duplication, inconsistency, and manual errors.
- Limited tracking, security, and analytical capabilities.

### 3. Alternate Systems

- Data managed through a single form or centralized record maintained by an executive.
- Heavy dependency on a single person for updates and management.
- Lack of scalability and high risk of data bottlenecks.
- Minimal automation and no real-time system-wide accessibility.
- Prone to delays, errors, and inefficiency in large institutions.

### 4. Proposed System: College ERP on Cloud

- Development of an integrated ERP system using **Django**.
- Cloud-based deployment for scalability, accessibility, and reliability.
- Modular structure supporting student management, inventory, event management, and public website.
- Centralized and normalized database for efficient data handling.
- Real-time updates and seamless integration across all components.
- Secure, role-based system ensuring data privacy and controlled access.

### 5. Objectives of the Proposed System

- To centralize and streamline all academic and administrative processes.
- To eliminate data redundancy and improve data accuracy through proper database design.

- To provide real-time, updated information across the institution.
- To enhance efficiency in managing large volumes of data and users.
- To enable easy deployment, monitoring, and maintenance using cloud infrastructure.
- To support data analysis for informed decision-making.
- To improve user experience through an intuitive and responsive interface.
- To build a scalable and future-ready system adaptable to technological advancements.

## Feasibility Study: College ERP System

The feasibility study of the proposed College ERP system evaluates its practicality across technical, economic, operational, and scalability aspects. The goal is to ensure that the system can be successfully developed, deployed, and maintained while meeting institutional requirements.

**Technical Feasibility:**
The system is technically feasible as it is built using proven and widely adopted technologies such as **Python** and the **Django**. These technologies provide a robust, scalable, and modular architecture, allowing easy integration of various components like student management, inventory, and event handling. The use of **PostgreSQL** ensures efficient data storage with strong support for normalization and data integrity. Deployment on cloud infrastructure using **Amazon Web Services** (AWS Free Tier initially) makes the system accessible and scalable. Additionally, containerization through **Docker** ensures consistency across development and production environments, reducing technical risks.

**Economic Feasibility:**
The project is economically viable as it primarily utilizes free and open-source technologies. Tools and frameworks such as Python, Django, PostgreSQL, and Docker significantly reduce licensing costs. Cloud deployment can begin with free-tier services, minimizing initial investment. The long-term cost benefits include reduced manual work, improved efficiency, and better resource utilization, making it a cost-effective solution, especially for institutions handling large-scale operations.

**Operational Feasibility:**
The proposed system is designed to be user-friendly and adaptable for students, faculty, and administrators. It replaces manual and semi-digital processes with an integrated platform, improving workflow efficiency. Features such as real-time updates, centralized data access, and role-based controls enhance usability and reliability. Training requirements are minimal due to intuitive UI design, and proper documentation ensures smooth adoption within the institution.

**Scalability and Future Feasibility:**
The system is highly scalable and future-ready. Cloud-based deployment allows handling increasing numbers of users and data without major infrastructure changes. The modular design enables easy addition of new features such as analytics, AI/ML integration, and mobile application support. This makes the system suitable not only for current requirements but also for future expansion.

## Cost and Benefit Analysis

The proposed College ERP system offers a highly favorable cost-to-benefit ratio by leveraging open-source technologies and cloud-based infrastructure. The initial costs are minimal, as development is carried out using free tools such as Python, the Django framework, and PostgreSQL, while deployment can begin on the free tier of Amazon Web Services. This significantly reduces expenses related to hardware, licensing, and infrastructure. In contrast, the benefits are substantial, including centralized data management, reduced manual workload, improved accuracy, real-time updates, and enhanced decision-making through data analysis. The system also improves operational efficiency, scalability, and user accessibility, particularly for institutions with a large number of students. Over time, the reduction in administrative overhead, errors, and redundant processes results in long-term cost savings, making the ERP system a cost-effective and high-value investment.

## Functional Requirements

In software engineering, a functional requirement defines a function of a software system or its component. A function is described as a set of inputs, the behavior, and outputs. Functional requirements may be calculations, technical details, data manipulation and processing and other specific functionality that show how a use case is to be fulfilled. They are supported by non-functional requirements, which impose constraints on the design or implementation (such as performance requirements, security, or reliability).As defined in requirements engineering, functional requirements specify particular behaviors of a system. This should be contrasted with non-functional requirements which specify overall characteristics such as cost and reliability.

## Technical Requirements:-

(i) Performance Requirements
(ii) Safety Requirements
(iii) Security Requirements
(iv) Hardware Constraints
(v) Software Constraints

(vi) Design Constraints

## Nonfunctional Requirements:-

In systems engineering and requirements engineering, non-functional requirements are requirements which specify criteria that can be used to judge the operation of a system, rather than specific behaviors. This should be contrasted with functional requirements that specify specific behavior or functions. Non-functional requirements are often called qualities of a system. Other terms for non-functional requirements are "constraints", "quality attributes", "quality goals" and "quality of service requirements". Qualities, of Non-functional requirements can be divided into two main categories. Execution qualities, such as security and usability, are observable at run time. Evolution qualities, such as extensibility and scalability, embody in the static structure of the software system.

## Software Quality Attributes

The main goals can be accomplished by standing true to the below main pillars :

(i) **Reliability:** Reliability will mostly depend on the client's connection status. The Systems Reliability solely relies upon its key files and folders.

(ii) **Availability:** The server on which this system will be running is expected to be available at all hours of the day to provide worldwide accessibility. A graphical user interface (GUI) is provided to have online interaction with the user.

(iii) **Maintainability:** The business logic here is that there can be accounting feature in the future development of the product.. All development will be provided with good documentation.

(iv) **Portability:** As the system is designed using Python, it can work on any platform Or architecture

# Chapter : 8

# Project Planning and Scheduling

# College ERP

## Project Planning & Scheduling:-

**Why (Agile context):**

- Agile focuses on sprints, timelines, and progress tracking → Gantt fits best.
- Easy to map 30-day / week-wise plan and visualize progress.
- Good for reporting and academic projects.

**Others (when useful):**

- **PERT Chart** → when task durations are uncertain (estimation heavy projects).
- **CPM (Critical Path Method)** → when dependency optimization and shortest path matter (complex, fixed workflows).

**Conclusion:**
For your Agile-based College ERP → **Gantt Chart is the most suitable** (simple, clear, aligns with sprint planning).

### Project Planning and Scheduling (30 Days – Week-wise)

**Week 1: Requirement Analysis & Planning**

- Finalize project scope and objectives
- Gather and document detailed requirements (modules: student, inventory, events, website)
- System analysis and feasibility confirmation
- Prepare system architecture and workflow diagrams
- Set up development environment (Python, Django, PostgreSQL)
- Initialize repository on GitHub

**Week 2: System Design & Initial Development**

- Design database schema with normalization
- Create Django project and apps (modular structure)
- Develop core backend models and basic APIs
- Start UI design (HTML, CSS, JS / optional React integration)
- Implement authentication (login, roles: admin, faculty, student)

**Week 3: Development & Integration**

- Complete major modules (student management, inventory, events)
- Integrate frontend with backend
- Implement CRUD operations and validations
- Add basic analytics/reporting features
- Begin testing (unit + integration testing)
- Containerize application using Docker

**Week 4: Testing, Deployment & Finalization**

- Perform system testing and bug fixing
- Optimize performance and database queries
- Deploy project on cloud using Amazon Web Services (free tier)
- Final UI improvements and responsiveness
- Documentation (user manual, technical report)
- Final review and project submission

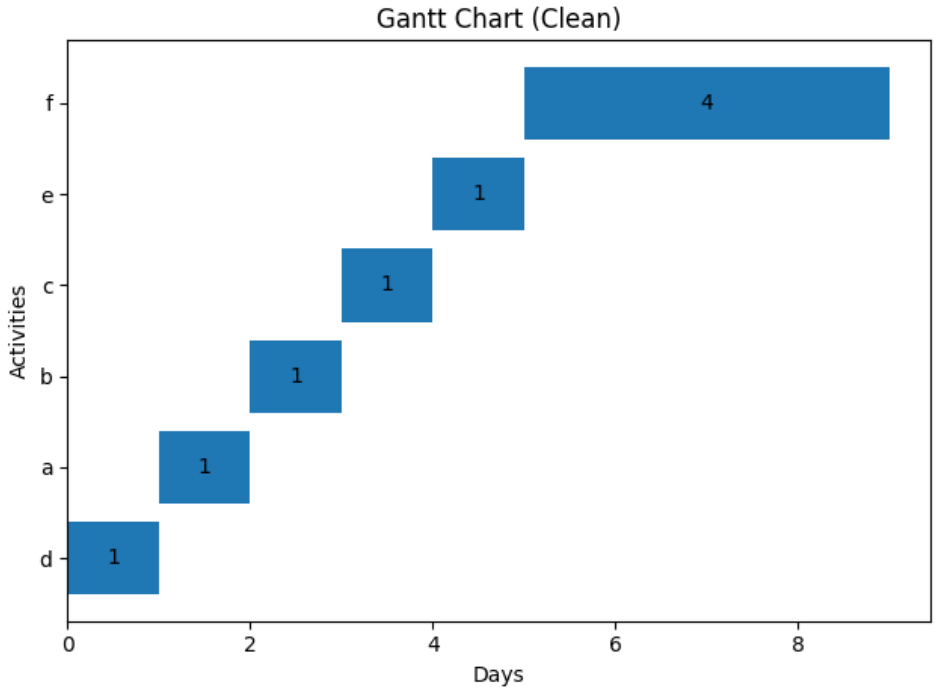
**Outcome:**
A fully functional, cloud-deployed College ERP system with core modules, tested features, and documented implementation within 30 days.

## 1. Gantt-Chart

Gantt Chart (Days, sequence: d → a → b → c → e → f)

| Activity | Duration | Start | End |
|---|---|---|---|
| **d (Pgsql)** | 1 | Day 0 | Day 1 |
| **a (Linux DevOps)** | 1 | Day 1 | Day 2 |
| **b (Shell + GitHub)** | 1 | Day 2 | Day 3 |
| **c (Docker)** | 1 | Day 3 | Day 4 |
| **e (CSS)** | 1 | Day 4 | Day 5 |
| **f (Django)** | 4 | Day 5 | Day 9 |

**Total = 9 days**



## 2. PERT CALCULATIONS (Correct)

Formula:

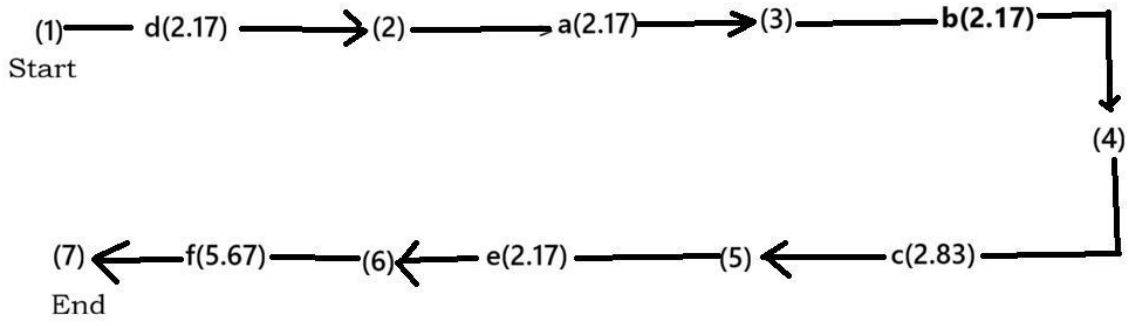
$$T_E = \frac{T_{opt} + 4T_{likely} + T_{pess}}{6}$$

**Expected Time (TE)**

| Activity | TOPT | TLIKELY | TPESS | TE |
|---|---|---|---|---|
| **a** | 1 | 2 | 4 | 2.17 |
| **b** | 1 | 2 | 4 | 2.17 |
| **c** | 1 | 3 | 4 | 2.83 |
| **d** | 1 | 2 | 4 | 2.17 |
| **e** | 1 | 2 | 4 | 2.17 |
| **f** | 4 | 6 | 6 | 5.67 |

**Total Project Time (PERT)**

**2.17+2.17+2.17+2.83+2.17+5.67=17.18 days**




2.17+2.17+2.17+2.83+2.17+5.67=17.18 days
PERT Digram

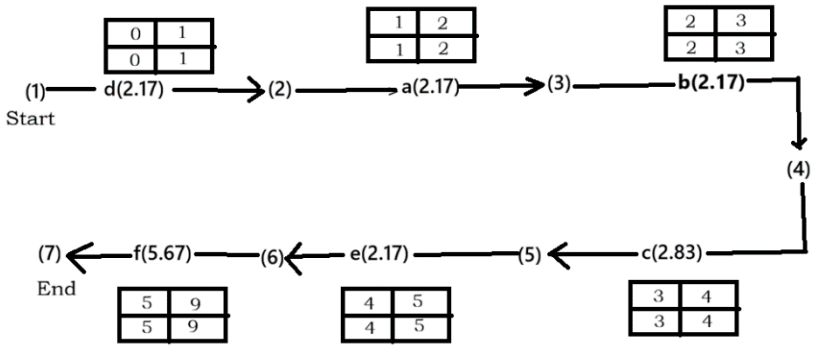
### 3. CPM (Critical Path Method)

Each activity must be shown as a **node box like this**:

```
  ES   |   EF
 ----  a  ----
  LS   |   LF
```

Using deterministic time:

| ACTIVITY | DUR | ES | EF | LS | LF | SLACK |
|---|---|---|---|---|---|---|
| D | 1 | 0 | 1 | 0 | 1 | 0 |
| A | 1 | 1 | 2 | 1 | 2 | 0 |
| B | 1 | 2 | 3 | 2 | 3 | 0 |
| C | 1 | 3 | 4 | 3 | 4 | 0 |
| E | 1 | 4 | 5 | 4 | 5 | 0 |
| F | 4 | 5 | 9 | 5 | 9 | 0 |




Critical Path Method Diagram
(CPM)

## FINAL CORRECT RESULTS

| Method | Duration |
|---|---|
| Gantt | 9 days |
| PERT | 17.18 days |
| CPM | 9 days |

- The project follows a **strict sequential (linear) workflow**, with each activity dependent on the previous one.
- The schedule is **simple, clear, and easy to manage**, but lacks flexibility due to no parallel tasks.
- Quality of scheduling is **accurate and predictable**, with well-defined start and end times.
- All activities lie on the **critical path**, so any delay directly impacts total completion time.

- Overall, the project management nature is **deterministic, tightly controlled, and dependency-driven**.

# Chapter : 9

# System Analysis

# College ERP

# Analysis : College ERP

**Introduction**
The College ERP System is designed as a centralized, cloud-based ,It replaces fragmented systems such as spreadsheets and manual processes with an integrated solution. The system aims to improve data accuracy, accessibility, security, and overall institutional productivity.

---

## Main Analysis Points

- **Existing System Issues**
  - Reliance on spreadsheets and manual records
  - Data redundancy and inconsistency
  - Lack of real-time access and collaboration
  - Poor scalability and dependency on network reliability
- **Proposed System Overview**
  - Cloud-based ERP with modular architecture
  - Centralized database for all departments
  - Web-based interface (HTML, CSS, JS / React)
  - Backend using Django framework
- **Core Functional Modules**
  - Student Management (admissions, profiles)
  - Faculty & Admin Management
  - Attendance & Examination System
  - Course & Timetable Management
  - Fee and Financial Management
- **Technical Architecture**
  - Backend: Django (modular apps, scalable design)
  - Database: PostgreSQL (structured, reliable storage)
  - Deployment: AWS (free tier for hosting)
  - Containerization: Docker for portability
  - Version Control: GitHub
- **System Requirements**
  - Internet connectivity for cloud access
  - Browser-based usage (cross-platform)
  - Secure authentication system (role-based access)
- **Security & Reliability**
  - Improved security over traditional PHP systems
  - Role-based access control (Admin, Faculty, Student)
  - Data backup and cloud reliability
- **Feasibility Considerations**
  - Technically feasible with modern stack
  - Economically viable using free-tier cloud services
  - Operationally efficient due to automation

---

Conclusion
The proposed College ERP system offers a robust, scalable, and secure solution to overcome the limitations of existing systems. With a modular architecture and modern technology stack, it ensures efficient management of institutional operations while supporting future growth and customization.

# Chapter : 10

# Use Case Diagrams

# College ERP

## Rent Management System : Use Case / Behaviour Diagram

### 1. Use Case Digrams (UML)

We use the Tool Use Case Model for analysis of behaviour of the System. It is a part of Unified Modelling Language. The Functional Requirements of the System and its interaction woth external agents (actors) are used. A Use Case Diagram gives us high level view of System without going in the details of the implementation.

The figure below illustrates the Use Case Model of Proposed System :

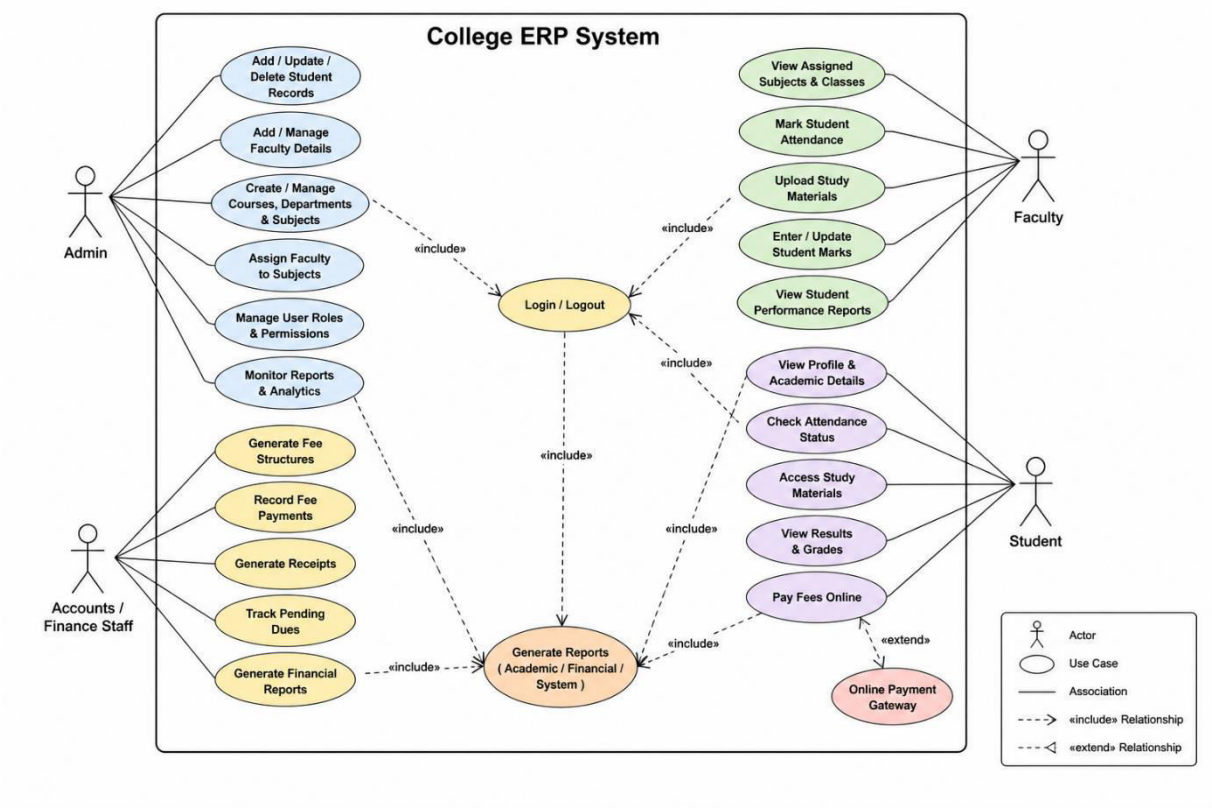



fig. USE CASE / Behaviour Digram for College ERP

---

## 2. Actors (Users)

i. **Admin**
ii. **Faculty**
iii. **Student**
iv. **Accounts/Finance Staff**

And We are free to add more on requirement , Since project is dynamic & customizable.

---

## 3. Key Use Cases (Point-wise)

### Admin Use Cases

i. **Login/Logout securely**
ii. **Add/Update/Delete student records**
iii. **Add/Manage faculty details**
iv. **Create courses, departments, and subjects**
v. **Assign faculty to subjects**
vi. **Monitor system reports and analytics**
vii. **Manage user roles and permissions**

---

### Faculty Use Cases

i. **Login to dashboard**
ii. **View assigned subjects and classes**
iii. **Mark student attendance**
iv. **Upload study materials/resources**
v. **Enter and update student marks**
vi. **View student performance reports**

---

### Student Use Cases

i. **Register/Login to system**
ii. **View personal profile and academic details**
iii. **Check attendance status**
iv. **Access study materials**
v. **View results and grades**
vi. **Pay fees online**

---

### Accounts/Finance Use Cases

- Generate fee structures
- Record fee payments
- Generate receipts
- Track pending dues
- Generate financial reports

---

## 4. Use Case Flow Example

### Use Case: Student Fee Payment

i. Actor: **Student**
ii. Pre-condition: **Student is logged in**
iii. Flow:
iv. **Student navigates to fee section**
v. **System displays pending fees**
vi. **Student selects payment option**
vii. **Payment is processed via gateway**
viii. **System updates payment status**
ix. **Receipt is generated**
x. Post-condition: **Fee status updated in database**

---

## 5. System Boundaries

i. Web-based/cloud-hosted ERP system
ii. Accessible via browser
iii. Integrated with database and payment gateway
iv. Supports modular architecture (Django apps / micro modules)

---

## 6. Benefits of Use Case Design

i. Clearly defines system functionality
ii. Helps in requirement validation
iii. Improves communication between stakeholders
iv. Simplifies development and testing

---

## 7. Conclusion

The use case study of the College ERP System outlines the interaction between users and the system, ensuring all functional requirements are well-defined. It serves as a blueprint for development, helping in building a scalable, modular, and efficient ERP solution.

# Chapter : 11

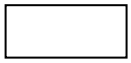
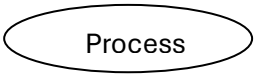
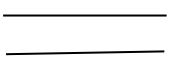
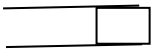
# DFD $0^{th}$ Level , Level-1 & Level-2

# College ERP

## Data Flow Diagram (DFDs):-

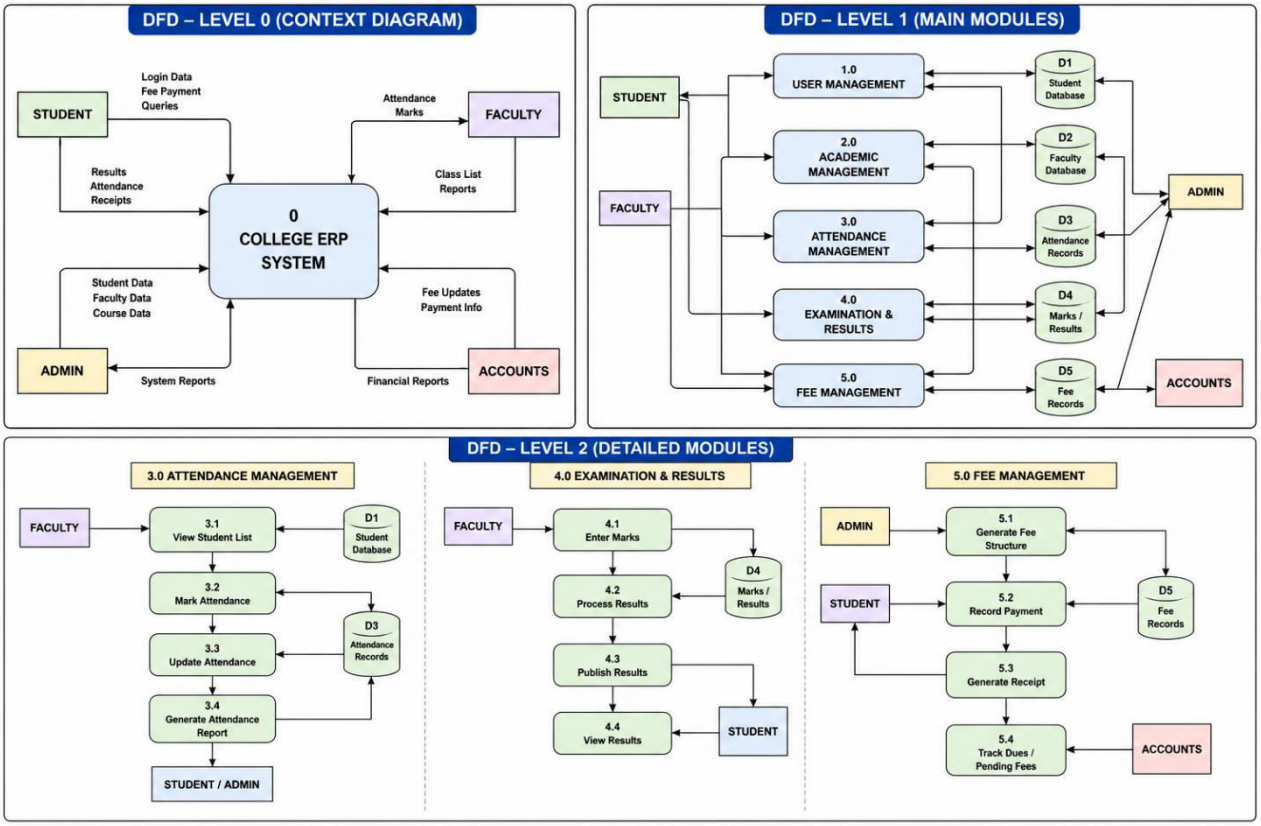
DFD stands for Data Flow Diagrams . This is also known as Bubble Charts through which we can represent the flow of data graphically in any information management system. We can easily understand the overall functionality of system because diagram represents the incoming dataflow and the outgoing dataflow and Stored data in the Graphical Form. It describes the flow of data in terms of input and output .

A DFD uses a number of notations or symbols that represent the flow of data in any System. These notations are illustrated as follows:

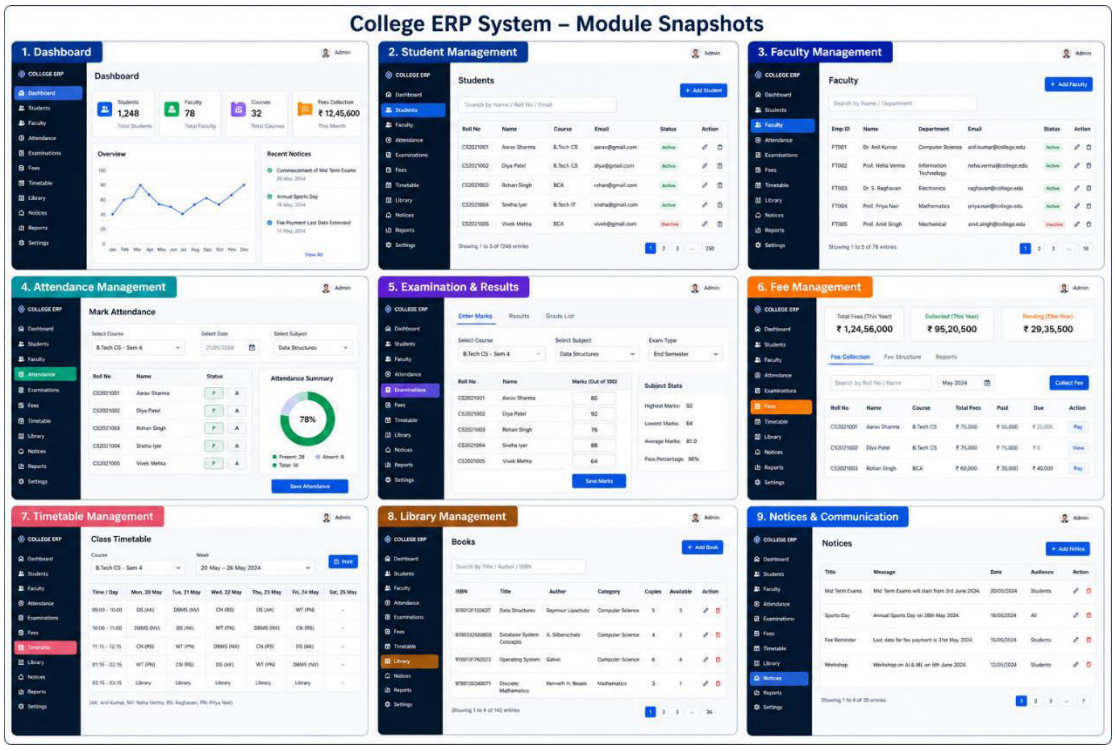
| Serial No. | Name of Symbol | Figure/Symbol | Purpose |
|---|---|---|---|
| 1 | Entity |  | A person / Entitity |
| 2 | Direction of Data Flow |  | Flow of Data from Entity/Process |
| 3 | Process or Bubble |   | Signify the process of a Module |
| 4 | Data Store |  OR  | Place of Storing Data |

The Rules We Follow while preparing the DFD's are as follows :

1. Each Process Should have atleast one input and one output.
2. Each Data Store should have atleast one data flow in and one data flow out.
3. All Process in DFD either go to another process or to another Data Store.




The above figure will be the helpful to make the outline of the GUI and made functional through the backend handled by the Django Framework.



# Chapter : 12

# Entity Relationship Diagrams & SQL Tables

# College ERP

## Introduction of Entity Relationship Diagrams of College ERP

The Entity Relationship (ER) Diagram is a fundamental data modeling technique used to visually represent the structure of a database system. In the context of a College ERP System, the ER diagram helps in identifying key entities such as Student, Faculty, Course, Attendance, and Fees, along with the relationships among them. It provides a clear blueprint for designing the database schema and ensures that data is organized efficiently.

An ER diagram consists of entities (real-world objects), attributes (properties of entities), and relationships (associations between entities). By using ER diagrams, developers can eliminate redundancy, maintain data integrity, and improve system scalability. It also acts as a communication tool between developers, analysts, and stakeholders by presenting complex database structures in a simplified graphical form.

In a College ERP system, ER diagrams play a crucial role in structuring modules like student management, academic records, examination systems, and financial operations. Each module is interconnected, and the ER model ensures that these interactions are logically defined.

**Note : Distributed Table Structure, with use of Normalization , controlled redundancy indexing for integrity and faster querying respectively.**

Overall, the ER diagram serves as the backbone of database design, enabling systematic development and efficient data handling in the ERP system.

# INTRODUCTION TO ER DIAGRAM

## 1. Introduction

An Entity-Relationship (ER) Diagram is a high-level conceptual data model that illustrates the structure of a database. It describes entities (real-world objects), their attributes (properties), and the relationships between them. ER diagrams are used in the requirement analysis and database design phases to provide a clear and effective communication between developers, stakeholders, and users.

In the context of College ERP, ER diagrams help in modeling academic and administrative data such as students, faculty, courses, attendance, examinations, fees, etc., ensuring data integrity and minimizing redundancy.

## 2. Components of ER Diagram

- **Entities:** Real-world objects or concepts (e.g., Student, Faculty)
- **Attributes:** Properties that describe an entity (e.g., RollNo, Name)
- **Relationships:** Associations between entities (e.g., Enrolls, Teaches)
- **Cardinality:** Defines the number of occurrences (1:1, 1:N, M:N)
- **Keys:** Primary Key uniquely identifies records in an entity

## 3. ER Diagram Symbols

| Symbol | Name | Description | Example |
|---|---|---|---|
|  | Entity | A real-world object or concept. | STUDENT |
|  | Attribute | Property or characteristic of an entity. | Name |
|  | Key Attribute | Attribute that uniquely identifies an entity. | RollNo |
|  | Relationship | Association between two or more entities. | Enrolls |
|  | Weak Entity | Entity that depends on another entity. | DEPENDENT |
|  | Identifying Relationship | Relationship used to identify a weak entity. | Has |
| Cardinality (Between Entities) | 1 —— | One to One | A 1 —— 1 B |
| | 1 —< | One to Many | A 1 —< B |
| | >—< | Many to Many | A >—< B |
|  | Derived Attribute | Attribute whose value i derived from other data. | Age |

## 4. Highlighting PostgreSQL

PostgreSQL

PostgreSQL is a powerful, open-source object-relational database management system (ORDBMS). It is known for its reliability, scalability, security and standards compliance.

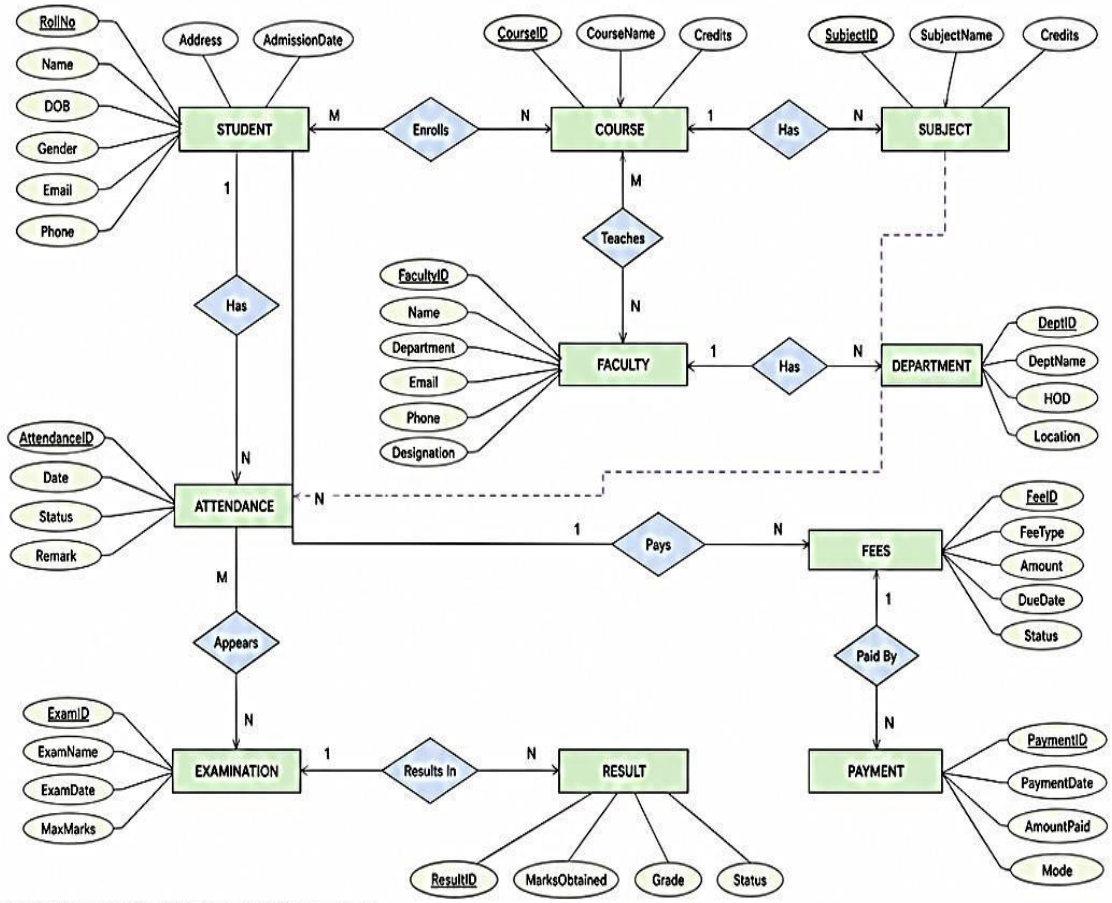
### Key Features

- ✓ Open Source and Free
- ✓ Strong ACID compliance
- ✓ Support for complex queries and joins
- ✓ Extensible data types and functions
- ✓ High performance and reliability
- ✓ Strong security and role management
- ✓ Support for JSON, XML and GIS data
- ✓ Cross-platform compatibility

### Why PostgreSQL for College ERP?

- ★ Handles large academic and administrative data efficiently.
- ★ Ensures data integrity and consistency.
- ★ Supports advanced reporting and analytics.
- ★ Cost-effective and widely supported by the developer community.

## 5. ER Diagram for College ERP (Selected Modules)




Note: PK attributes are underlined. Relationships and cardinalities represent typical constraints used in College ERP. Actual implementation may vary as per requirements.

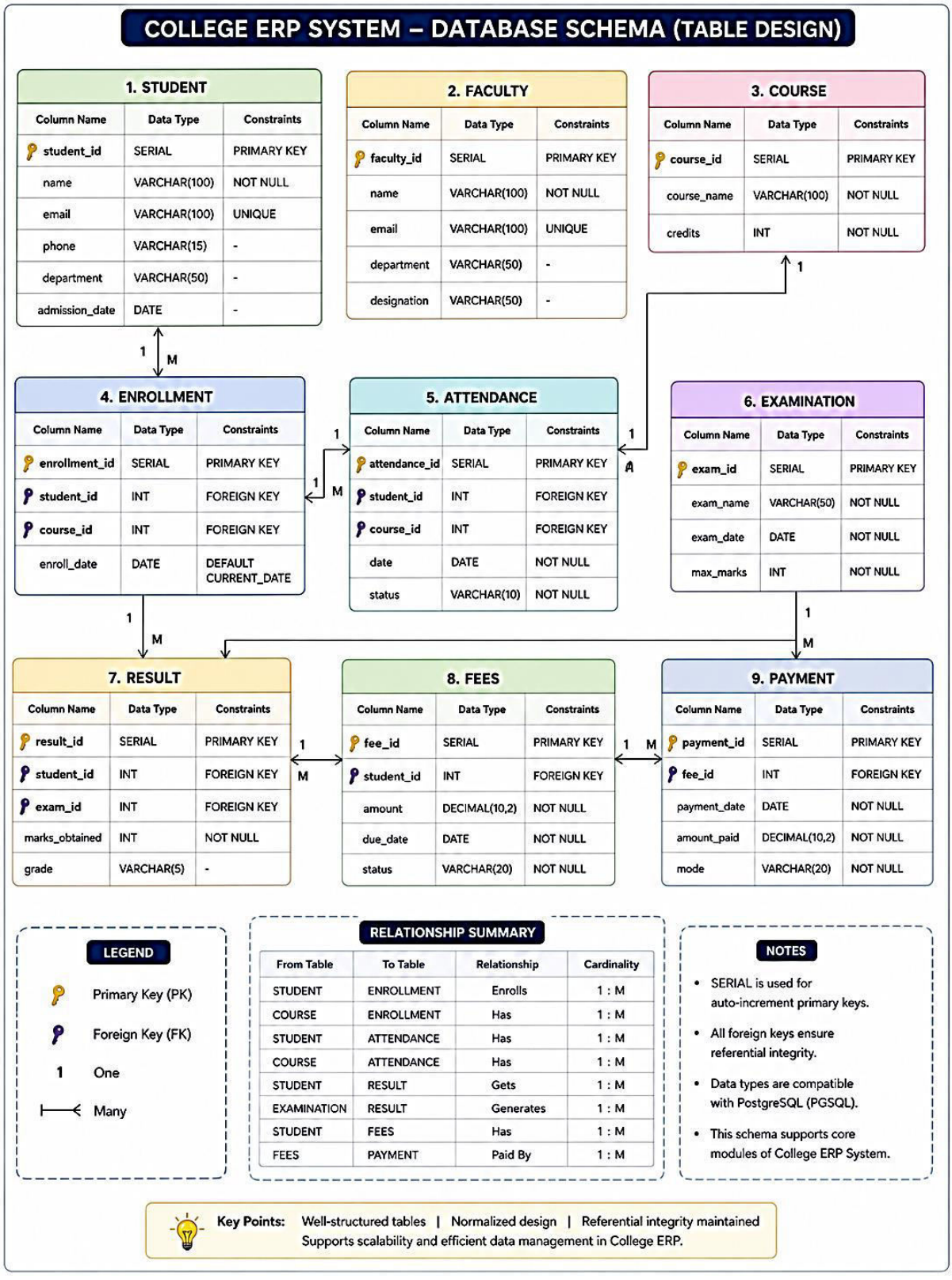



**Fig. Schema of Table Design (convertible to sql database tables)**

# Chapter : 13

# Future Scope of The System

# College ERP

# Future Scope of the College ERP

The future scope of the College ERP system developed using Django, AWS, Docker, and PostgreSQL is highly scalable, flexible, and innovation-driven. The system can evolve from a basic institutional management platform into a fully integrated digital ecosystem supporting academic, administrative, and analytical needs.

In the future, the ERP can be enhanced with **AI-based analytics and prediction systems** to monitor student performance, attendance trends, and dropout risks. This will help institutions take proactive decisions. Integration of **machine learning models** can also enable personalized learning recommendations for students.

The system can be extended with **mobile application support (Android/iOS)** to provide real-time access to students, faculty, and parents. Features like notifications, assignment tracking, attendance updates, and fee reminders can be seamlessly delivered via mobile platforms.

With AWS infrastructure, the project can scale to support **multi-college or university-level deployments**, enabling centralized management of multiple campuses. Features like **multi-tenant architecture** can be implemented so that different institutions can use the same system securely with isolated data.

The use of Docker allows future improvements in **microservices architecture**, where different modules (admissions, exams, finance, HR) can run independently, improving maintainability and deployment efficiency. Continuous Integration/Continuous Deployment (CI/CD) pipelines can also be introduced for automated testing and faster updates.

Further enhancements can include **integration with third-party services**, such as payment gateways for online fee submission, biometric systems for attendance, and external APIs for verification processes. Cloud services like AWS can also enable **data backup, disaster recovery, and high availability** mechanisms.

Advanced modules such as **Learning Management System (LMS), online examination system, and virtual classrooms** can be incorporated to support digital education trends. Additionally, role-based dashboards with real-time reporting can improve decision-making at all administrative levels.

In conclusion, the College ERP system has strong future potential to become a comprehensive, intelligent, and scalable platform. With continuous upgrades and integration of modern technologies, it can significantly enhance the efficiency, transparency, and digital transformation of educational institutions.

# Chapter : 14

# Limitations of the Project

# College ERP

## Limitations of the College ERP

The College ERP system developed using Django, AWS, Docker, and PostgreSQL provides a modern, scalable, and efficient solution for managing academic and administrative operations. It significantly improves data centralization, reduces manual effort, enhances transparency, and supports real-time access to information. Features like modular design, cloud deployment, and containerization ensure flexibility, security, and easy scalability. Overall, the system delivers strong value by streamlining workflows, improving decision-making, and enabling digital transformation in educational institutions.

Despite these advantages, certain limitations exist; however, they are manageable and are effectively mitigated through design choices and future scalability provisions:

**Limitations (with controlled impact):**

- **Initial Setup Complexity:** Deployment using Docker and AWS may require technical expertise, but once configured, it becomes highly stable and reusable.
- **Internet Dependency:** Cloud-based access requires reliable internet connectivity; however, this ensures real-time data synchronization and remote accessibility.
- **Cost of Cloud Services:** AWS usage may incur costs over time, but efficient resource management and scaling options help optimize expenses.
- **User Training Requirement:** Faculty and staff may need initial training, though the system's intuitive interface reduces the learning curve significantly.
- **Data Security Concerns:** Storing data on the cloud raises concerns, but AWS security features and role-based access control minimize risks.
- **Customization Effort:** Highly specific institutional requirements may need additional development, yet Django's modular nature makes customization feasible.
- **System Maintenance:** Regular updates and monitoring are required, but containerization (Docker) simplifies maintenance and deployment processes.

In conclusion, while the system has some limitations, they are outweighed by its benefits and are effectively addressed through modern technologies and design strategies. The ERP remains a robust, future-ready solution capable of continuous improvement and expansion.

# Chapter : 15

# References and Learning Resources

# College ERP

References and Learning Resources for the Project : College ERP

- Official documentation of Django Framework – used for backend development, project structuring, and module integration.
- Official documentation of Docker – referred for containerization, image creation, and deployment workflows.
- Official documentation of AWS (Amazon Web Services) – used for cloud deployment, hosting, and scalability concepts.
- PostgreSQL official documentation – referred for database design, queries, and data management.
- Various online tutorials and technical videos (YouTube) on Django, Docker, and AWS – used for practical implementation and troubleshooting.
- Workshop materials and hands-on training sessions on Cloud Computing and Web Development – contributed to applied learning and project execution.
- GitHub repositories and community discussions – referred for code practices, issue resolution, and version control understanding.

**Note:**
The development of this project was primarily supported through practical exposure, workshop-based learning, and official technical documentation, ensuring a hands-on and implementation-oriented approach.

# Chapter : 16

# Certifications of Developer For Making Project

# College ERP

# Certifications of Developer For Making Project

Learner **Aayush Sahay** had undergone workshops **for skill and practical exposure** for the following workshops for the durations held by the organizations / institutes as follows:

| Sno. | Workshop / FDP | Institute / Organization | Topic of Certication | Issue Date |
|---|---|---|---|---|
| **1** | FDP | IIT Guwahati | Python for Engineering Applications | 08/08/25 |
| **2** | Workshop | CCDTE, PATNA | Cloud Computing with AWS | 14/08/25 |
| **3** | Workshop | CCDTE, PATNA | Django and Docker Workshop | 13/10/25 |

Digital India
Power To Empower

सत्यमेव जयते

ELECTRONICS INDIA
Billion Needs Million Chips

Ref. no: EICT/P-II/JFDP/25-26/010/387

**Electronics & ICT Academy**

*Supported by Ministry of Electronics and Information Technology (MeitY), Govt. of India*

**Indian Institute of Technology Guwahati**

# Participation Certificate

**This is to certify that**

**Mr./Ms./Dr.** Aayush Sahay

of CIMAGE Center of Digital Technology & Entrepreneurship

has participated in a two - weeks Online Joint Faculty Development Programme on **"Python Programming for Engineering Applications"** jointly organised by Electronics and ICT Academies held from **28 July - 08 August, 2025** under the **"Scheme of financial assistance for setting up of Electronics and ICT Academies"** of the **Ministry of Electronics and Information Technology (MeitY), Government of India**. He/she has also taken part in the hands on sessions during the program. This Faculty Development Programme is at par with other Quality Improvement Programmes*.

| | | | | | |
|---|---|---|---|---|---|
| **Dr. Aryabartta Sahu**<br>Principal Coordinator<br>IIT Guwahati<br>Assam | **Prof. Atul Gupta**<br>Joint Principal Coordinator<br>IIITDM Jabalpur<br>Madhya Pradesh | **Prof. Partha Pratim Roy**<br>Joint Principal Coordinator<br>IIT Roorkee<br>Uttarakhand | **Prof. Sanjeev Manhas**<br>oint Principal Coordinator<br>IIT Roorkee<br>Uttarakhand | **Dr. Bhaskar Mondal**<br>Joint Principal Coordinator<br>NIT Patna<br>Bihar | **Dr. Prasanta Majumdar**<br>Joint Principal Coordinator<br>MNIT Jaipur<br>Rajasthan |

** As per minutes of the 3rd PRSG.*

CIMAGE
Knowledge . Skill . Success
CIMAGE GROUP OF INSTITUTIONS

amazon web services

# CERTIFICATE
## OF PARTICIPATION

This is to certify that

Aayush Sahay

has actively participated in the workshop on **Cloud Computing with AWS** organized by **CIMAGE Professional College**.

The workshop was conducted from **24th July, 2025 to 25th July, 2025**, with a total duration of **10 hours**. The program provided hands-on learning and practical insights into AWS cloud services, enhancing the participant's technical knowledge and skills.

We acknowledge and appreciate the enthusiasm and dedication shown during the workshop.

Coordinator

Director

Certificate No: 1062/2025/4971

Issued Date: 14th-Aug-2025

CIMAGE
Knowledge . Skill . Success
CIMAGE GROUP OF INSTITUTIONS

django

# Certificate

## OF PARTICIPATION

This is to certify that

Aayush Sahay

has successfully participated in the **Django Workshop** conducted from **26th July to 27th July 2025**, organized by **Catalyst College**.

The workshop offered hands-on experience in **Django** and enhanced participants' understanding of web development.

We appreciate the participant's enthusiasm and active involvement throughout the workshop.

Certificate No.: 1062/2025/5081

Issued Date: 13th-Oct-2025

Coordinator

Director

# College ERP — Deployment & Setup Documentation

Plain-text export of Docker & environment setup files.

## Index

# 1. Dockerfile

Dockerfile

```
FROM python:3.13-slim

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir -r pip_dependency.txt

EXPOSE 8000

CMD ["python","manage.py","runserver","0.0.0.0:8000"]
```

# 2. How to setup Vm.txt

How_to_setup_Vm.txt

```
connect via ssh:
ssh -i "CollegeERP_Server_Key.pem" ubuntu@ec2-3-110-193-190.ap-south-1.compute.amazonaws.com

--success in connecting to vm

update system
sudo apt update
sudo apt install git -y
git clone https://github.com/AAYUSH-SAHAY-24326711019/ERP_College_Django.git

installation of docker

sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
newgrp docker
docker pull postgres:15


creating the container of the postgres

docker run --name erp_db_pgsql -e POSTGRES_PASSWORD=root -p 5432:5432 -d postgres:15

access the container
docker exec -it erp_db_pgsql psql -U postgres

copy the sql file to the home.
make the sql dump to the database
docker exec -i erp_db_pgsql psql -U postgres -d erp_db < ~/erp_db.sql

changing the settings.py (done)
now making the image from the configured repo.
docker build -t erpcollege_app .

check the running ones
docker ps -a
stop the postgres container
docker stop <id>

create the network
docker network create erp_network
docker network ls

inspect the network

docker network inspect erp_network

connect the pgsql container to the network
docker network connect erp_network erp_db_pgsql

start the postgres container and check the erp_network
found the pgsql container over there.

make the container from erpcollege_app
+ Adding to erp_network
docker run --name erpcollege_container --network erp_network -p 8000:8000 -d erpcollege_app

added to the network
see for the errors :
```

```
docker logs -f erpcollege_container
(no errors ! success)

docker exec -it erpcollege_container python manage.py shell
go to location:
exec(open("make_user_script.py").read())
quit()

time to make the credentials.
```

# 3. pip_dependency.txt

pip_dependency.txt

```
asgiref==3.11.1
Django==6.0.3
pillow==12.2.0
psycopg2-binary==2.9.12
sqlparse==0.5.5
tzdata==2025.3
djangorestframework==3.17.1
djangorestframework_simplejwt==5.5.1
```

# College ERP (Django) — Source Code

Plain-text export of key application files for reference / review.

## Index

# 1. Project Configuration (erpCollege)

## settings.py

erpCollege/settings.py

```
"""
Django settings for erpCollege project.

Generated by 'django-admin startproject' using Django 5.2.4.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/
"""
import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = "django-insecure-q6isb5l6=r^1nntg%63o=72drmxzuay5$5ma7v72xlv*(up@x_"

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['*']


# Application definition

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "rest_framework",
    "appStudent",
    "appFaculty",
    "appHOD",
    "appMainsite",
    "appErpAdmin",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

ROOT_URLCONF = "erpCollege.urls"
```

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [os.path.join(BASE_DIR,'templates')],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]

WSGI_APPLICATION = "erpCollege.wsgi.application"


# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'erp_db',
        'USER': 'admin_erp_db',
        'PASSWORD': 'abcdef',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}


# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.CommonPasswordValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.NumericPasswordValidator",
    },
]


# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/

LANGUAGE_CODE = "en-us"

TIME_ZONE = "UTC"

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/
```

```
STATIC_URL = "static/"
STATICFILES_DIRS =[
    os.path.join(BASE_DIR,'static')
]

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR/ 'media'
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

## urls.py (root)

erpCollege/urls.py

```
"""
URL configuration for erpCollege project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/5.2/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""

from django.contrib import admin
from django.urls import path,include
import appStudent.views
import appFaculty.views
import appHOD.views
import appMainsite.views
import appErpAdmin.views
from django.conf import settings
from django.conf.urls.static import static


urlpatterns = [
    path("admin/", admin.site.urls),
    path('', appMainsite.views.mainsite, name="index"),
    path('loghod/', appHOD.views.loghod, name="loghod"),
    path('erpadmin/', include('appErpAdmin.urls')),
    path('students/', include('appStudent.urls')),
    path('faculty/',include('appFaculty.urls'))

]

urlpatterns += static(
    settings.MEDIA_URL,
    document_root=settings.MEDIA_ROOT
)
```

# 2. App: appErpAdmin

## admin.py

appErpAdmin/admin.py

```
from django.contrib import admin

# Register your models here.
```

## forms.py

appErpAdmin/forms.py

```
from django import forms
from appStudent.models import Student
from appFaculty.models import Faculty
from django.contrib.auth.models import User

class Form1_main(forms.ModelForm):
    class Meta:
        model = Student

        fields =[
            "id",
            "name",
            "student_id",
            "course",
            "email",
            "session",

        ]
        def save(self, commit=True):
            student = super().save(commit=False)

            # create user only if not linked already
            if not student.user:
                user, created = User.objects.get_or_create(
                username=student.student_id
                )

                if created:
                    user.set_password(student.student_id)
                    user.save()

            student.user = user

            if commit:
                student.save()

            return student

class Form2_main(forms.ModelForm):
    class Meta:
        model = Faculty

        fields =[
            "faculty_id",
            "faculty_name",
            "faculty_email",
            "optionselected",
            "faculty_designation",

        ]
```

```
        def save(self, commit=True):
            faculty = super().save(commit=False)

            # create user only if not linked already
            if not faculty.user:
                user, created = User.objects.get_or_create(
                username=faculty.faculty_id
                )

                if created:
                    user.set_password(faculty.faculty_id)
                    user.save()

            faculty.user = user

            if commit:
                faculty.save()

            return faculty
```

## models.py

appErpAdmin/models.py

```
from django.db import models
from django.contrib.auth.models import User

# Create your models here.

class AdminRoles(models.Model):
    role_id = models.AutoField(primary_key=True)

    role = models.CharField(max_length=100,unique=True)

    def __str__(self):
        return self.role

class ErpAdmin(models.Model):
    id=models.AutoField(primary_key=True)
    # jwt code add user field
    user = models.OneToOneField(
    User,
    on_delete=models.CASCADE,
    null=True
    )
    #

    role = models.ForeignKey(
        AdminRoles,
        on_delete=models.CASCADE,
        db_column='role_id'
    )

    name = models.CharField(max_length=150)

    email = models.EmailField(unique=True)

    password = models.CharField(max_length=255)

    def __str__(self):
        return self.name


class Courses(models.Model):
    id = models.AutoField(primary_key=True)
```

```
    COURSE_CHOICES = [
        ('BCA', 'BCA'),
        ('BSC IT', 'BSC IT'),
        ('BBA', 'BBA'),
        ('BCOM', 'BCOM'),
        ('PGDM', 'PGDM'),
        ('MBA', 'MBA'),
        ('MCA', 'MCA'),
    ]

    course_name = models.CharField(
        max_length=100,
        choices=COURSE_CHOICES,
        unique=True
    )

    def __str__(self):
        return self.course_name


class University(models.Model):
    id = models.AutoField(primary_key=True)

    UNIVERSITY_CHOICES = [
        ('AKU', 'AKU'),
        ('PPU', 'PPU'),
    ]

    university_name = models.CharField(
        max_length=100,
        choices=UNIVERSITY_CHOICES,
        unique=True
    )

    def __str__(self):
        return self.university_name

class CourseSessions(models.Model):
    id = models.AutoField(primary_key=True)

    course = models.ForeignKey(
        Courses,
        on_delete=models.CASCADE
    )

    university = models.ForeignKey(
        University,
        on_delete=models.CASCADE
    )

    start_year = models.IntegerField()

    end_year = models.IntegerField()

    complete_name = models.CharField(
        max_length=255,
        unique=True
    )

    def save(self, *args, **kwargs):
        if self.course_id and self.university_id:
            self.complete_name =
f"{self.course.course_name}_{self.university.university_name}_{self.start_year}-{self.end_year}"
            super().save(*args, **kwargs)
```

```
    def __str__(self):
        return self.complete_name


class StudentEnrollment(models.Model):
    student = models.ForeignKey('appStudent.Student',on_delete=models.CASCADE)
    course = models.ForeignKey('appErpAdmin.CourseSessions',on_delete=models.CASCADE)
```

## urls.py

appErpAdmin/urls.py

```
from django.urls import path
from . import views

urlpatterns = [

    # ========================
    # ADMIN LOGIN
    # ========================
    path(
        '',
        views.admin_login,
        name='admin_login'
    ),

    path(
        'logout/',
        views.admin_logout,
        name='admin_logout'
    ),

    # ========================
    # CSV DOWNLOAD
    # ========================
    path(
        'download-mainsite-csv/',
        views.download_mainsite_csv,
        name='download_mainsite_csv'
    ),

    # ========================
    # COURSE SESSION
    # ========================
    path(
        'course-session-page/',
        views.course_session_page,
        name='course_session_page'
    ),

    path(
        'save-course-session/',
        views.save_course_session,
        name='save_course_session'
    ),


    path('show-student-course/',views.showStudentCourse,name='showStudentCourse'),

    path('form1/',views.form1,name='form1'),
    path('form1_all/',views.form1_all,name='form1_all'),
    path('form1_edit/<int:id>',views.form1_edit,name='form1_edit'),

    path('form2/',views.form2,name='form2'),
    path('form2_all/',views.form2_all,name='form2_all'),
```

```
    path('form2_edit/<int:id>',views.form2_edit,name='form2_edit'),

]
```

## views.py

appErpAdmin/views.py

```
import json
from django.http import HttpResponse,JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.shortcuts import render,redirect,get_object_or_404
from .models import ErpAdmin
from appMainsite.models import MainsiteEnquiryForm
from appStudent.models import Student
from appErpAdmin.models import Courses,University,CourseSessions,StudentEnrollment
from appFaculty.models import Faculty
import csv
import calendar
from datetime import datetime
from django.utils.timezone import now
from django.contrib.auth import authenticate
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.authentication import JWTAuthentication
from .forms import Form1_main,Form2_main




# Create your views here.



def admin_login(request):


    # =========================
    # LOGIN PROCESS
    # =========================

    if request.method == "POST":

        admin_email = request.POST.get('admin_email')

        password = request.POST.get('password')

        try:

            admin = ErpAdmin.objects.get(
                email=admin_email,
                password=password
            )

            user = authenticate(
            username=admin_email,
            password=password
            )

            if user is not None:
                # jwt code
                refresh = RefreshToken.for_user(admin)
                access_token = str(refresh.access_token)
                # print(access_token)
```

```
            request.session['access_token'] = access_token
            #

            token = request.session.get('access_token')
            # SESSION
            request.session['admin_id'] = admin.id
            request.session['admin_name'] = admin.name
            request.session['admin_role'] = admin.role.role
            if token:
                jwt_auth = JWTAuthentication()
                validated_token = jwt_auth.get_validated_token(token)
                # ========================
                # DASHBOARD DATA
                # ========================

                current_date = datetime.now()

                year = current_date.year

                month = current_date.month

                enquiries = MainsiteEnquiryForm.objects.filter(
                    created_at__year=year,
                    created_at__month=month
                ).order_by('-id')

                months = []

                for i in range(1, 13):

                    months.append({
                        'number': i,
                        'name': calendar.month_name[i]
                    })

                courses = Courses.objects.all()
                universities = University.objects.all()
                current_courses = CourseSessions.objects.all()

                context = {

                    'enquiries': enquiries,

                    'months': months,

                    'current_month': month,

                    'current_year': year,

                    'courses': courses,

                    'universities': universities,

                    'current_courses':current_courses,

                }

                return render(
                    request,
                    'erpadmin/dashboard.html',
                    context
                )

    except ErpAdmin.DoesNotExist:
```

```
        return render(
            request,
            'erpadmin/index.html',
            {
                'error': 'Invalid Credentials'
            }
        )



# =========================
# IF NOT LOGGED IN
# =========================

if 'admin_id' not in request.session:

    return render(
        request,
        'erpadmin/index.html'
    )



# =========================
# DASHBOARD FILTERING
# =========================

current_date = datetime.now()

year = int(
    request.GET.get(
        'year',
        current_date.year
    )
)

month = int(
    request.GET.get(
        'month',
        current_date.month
    )
)

enquiries = MainsiteEnquiryForm.objects.filter(
    created_at__year=year,
    created_at__month=month
).order_by('-id')

months = []

for i in range(1, 13):

    months.append({
        'number': i,
        'name': calendar.month_name[i]
    })

courses = Courses.objects.all()
universities = University.objects.all()
context = {

    'enquiries': enquiries,

    'months': months,
```

```
        'current_month': month,

        'current_year': year,

        'courses': courses,

        'universities': universities,

    }

    if request.GET.get('ajax'):

        return render(
            request,
            'erpadmin/partials/mainsite_enquiry_rows.html',
            context
        )

    return render(
        request,
        'erpadmin/dashboard.html',
        context
    )



# ========================================
# CSV DOWNLOAD VIEW
# ========================================

def download_mainsite_csv(request):

    if 'admin_id' not in request.session:

        return redirect('admin_login')

    current_date = datetime.now()

    year = int(
        request.GET.get(
            'year',
            current_date.year
        )
    )

    month = int(
        request.GET.get(
            'month',
            current_date.month
        )
    )

    enquiries = MainsiteEnquiryForm.objects.filter(
        created_at__year=year,
        created_at__month=month
    ).order_by('-id')

    response = HttpResponse(content_type='text/csv')

    filename = f"mainsite_enquiries_{month}_{year}.csv"

    response['Content-Disposition'] = f'attachment; filename="{filename}"'

    writer = csv.writer(response)
```

```
    writer.writerow([
        'Name',
        'Mobile',
        'From Place',
        'Course',
        'Created At'
    ])

    for enquiry in enquiries:

        writer.writerow([
            enquiry.name,
            enquiry.mobile,
            enquiry.from_place,
            enquiry.course,
            enquiry.created_at
        ])

    return response


def admin_logout(request):

    request.session.flush()

    return redirect('admin_login')

def course_session_page(request):

    courses = CourseSessions.objects.all()
    universities = University.objects.all()

    sessions = CourseSessions.objects.select_related(
        'course', 'university'
    ).all()

    return render(request, 'course_session.html', {
        'courses': courses,
        'universities': universities,
        'sessions':sessions,
    })

def save_course_session(request):

    if request.method == "POST":

        course_id = request.POST.get('course')
        university_id = request.POST.get('university')
        start_year = request.POST.get('start_year')
        end_year = request.POST.get('end_year')

        course = Courses.objects.get(id=course_id)
        university = University.objects.get(id=university_id)

        obj = CourseSessions.objects.create(
            course=course,
            university=university,
            start_year=start_year,
            end_year=end_year
        )

        data = {
            'id': obj.id,
            'complete_name': obj.complete_name
        }
```

```
        return JsonResponse(data)

    return JsonResponse({'error': 'Invalid Request'})


from .models import CourseSessions

def showRecords(request):
    records = CourseSessions.objects.all()

    return render(request,'erpadmin/student_enrollment.html',
                  {'records':records})


# def seeCurrentCourses(request):


def showStudentCourse(request):

    if request.method =="POST":
        studentid = request.POST.get("student_id")
        id_list = [int(x.strip()) for x in studentid.split(",")]
        courseid = request.POST.get("course_id")

        for i in id_list:
            x=courseid
            student_obj = Student.objects.get(id=i)
            course_obj = CourseSessions.objects.get(id=x)
            StudentEnrollment.objects.create(student=student_obj,course=course_obj)

        data=StudentEnrollment.objects.all()
        context1={
            'data':data
        }

        return render(request, 'erpadmin/success.html',context1)

    return redirect('dashboard')


def form1(request):
    if request.method=="POST":
        form = Form1_main(request.POST)
        if form.is_valid():
            form.save()
            request,
            'erpadmin/dashboard.html'
    else:
        form = Form1_main()
    return render(request,'erpadmin/add_students_form1.html',{"form":form})

def form1_all(request):
    students=Student.objects.all().order_by('id')
    return render(request,'erpadmin/all_students_form1.html',{"students":students})

def form1_edit(request,id):
    student=get_object_or_404(Student,id=id)
    if request.method=="POST":
        form = Form1_main(request.POST,instance=student)

        if form.is_valid():
            form.save()
            return redirect('/erpadmin/form1_all')
    else:
```

```
        form =Form1_main(instance=student)
    return render(request,'erpadmin/edit_student_form1.html',{'form':form})


def form2(request):
    if request.method=="POST":
        form = Form2_main(request.POST)
        if form.is_valid():
            form.save()
            request,
            'erpadmin/dashboard.html'
    else:
        form = Form2_main()
    return render(request,'erpadmin/add_faculty_form2.html',{"form":form})

def form2_all(request):
    faculty=Faculty.objects.all().order_by('id')
    return render(request,'erpadmin/all_faculty_form2.html',{"faculty":faculty})

def form2_edit(request,id):
    faculty=get_object_or_404(Faculty,id=id)
    if request.method=="POST":
        form = Form2_main(request.POST,instance=faculty)

        if form.is_valid():
            form.save()
            return redirect('/erpadmin/form2_all')
    else:
        form =Form2_main(instance=faculty)
    return render(request,'erpadmin/edit_faculty_form2.html',{'form':form})
```

# 3. App: appFaculty

## admin.py

appFaculty/admin.py

```
from django.contrib import admin
from .models import FacultyWiseAttendance
from .models import StudentWiseAttendance
# Register your models here.

admin.site.register(FacultyWiseAttendance)
admin.site.register(StudentWiseAttendance)
```

## models.py

appFaculty/models.py

```
from django.db import models
from django.contrib.auth.models import User

class Faculty(models.Model):

    DESIGNATION_CHOICES = [
        ('Professor', 'Professor'),
        ('Guest Lecturer', 'Guest Lecturer'),
        ('Assistant Professor', 'Assistant Professor'),
        ('HOD - IT', 'HOD - IT'),
        ('HOD - Management', 'HOD - Management'),
        ('IT / Skill Trainer', 'IT / Skill Trainer'),
    ]

    # jwt code add user field
    user = models.OneToOneField(
    User,
    on_delete=models.CASCADE,
    null=True
    )
    #

    date_created = models.DateTimeField(auto_now_add=True)

    faculty_email = models.EmailField(unique=True)

    faculty_name = models.CharField(max_length=100)

    faculty_id = models.CharField(
        max_length=10,
        unique=True
    )

    faculty_designation = models.CharField(
        max_length=50,
        choices=DESIGNATION_CHOICES
    )

    optionselected = models.CharField(max_length=20)

    def __str__(self):
        return f"{self.faculty_name} ({self.faculty_id})"


class ActivityLogsFaculty(models.Model):

    ACTION_CHOICES = [
```

```
        ('Login', 'Login'),
        ('Logout', 'Logout'),
    ]

    faculty = models.ForeignKey(
        Faculty,
        on_delete=models.CASCADE
    )

    action = models.CharField(
        max_length=10,
        choices=ACTION_CHOICES
    )

    timestamp = models.DateTimeField(
        auto_now_add=True
    )

    def __str__(self):
        return f"{self.faculty.faculty_id} - {self.action}"


class FacultyAssignedSubject(models.Model):

    faculty = models.ForeignKey(
        Faculty,
        on_delete=models.CASCADE,
        to_field='faculty_id',
        db_column='faculty_id',
        related_name='assigned_subjects'
    )

    subject_name = models.CharField(max_length=200)

    subject_code = models.CharField(max_length=20)

    semester = models.CharField(max_length=20)

    session = models.CharField(max_length=20)

    date_assigned = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.faculty.faculty_name} - {self.subject_name}"

# --
class FacultyWiseAttendance(models.Model):

    attendance_date = models.DateField()

    course_name = models.CharField(max_length=50)

    faculty_id = models.CharField(max_length=20)

    faculty_name = models.CharField(max_length=100)

    subject = models.CharField(max_length=200)

    attendance = models.TextField()

    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.course_name} - {self.subject} - {self.attendance_date}"
```

```
class StudentWiseAttendance(models.Model):

    attendance_date = models.DateField()

    course_name = models.CharField(max_length=50)

    subject = models.CharField(max_length=200)

    faculty_id = models.CharField(max_length=20)

    student_id = models.CharField(max_length=20)

    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.student_id} - {self.subject}"
# --
```

## urls.py

appFaculty/urls.py

```
from django.urls import path
from .import views


urlpatterns = [
    path('', views.faculty_login, name='faculty_login'),
    path('logout/', views.faculty_logout, name='faculty_logout'),
    path('makeSchedulesIT/',views.makeSchedulesIT,name='makeSchedulesIT'),
    path('makeSchedulesM/',views.makeSchedulesM,name='makeSchedulesM'),
    path('add-faculty-subject/',views.add_faculty_subject,name='add_faculty_subject'),
    path('mark-attendance/',views.mark_attendance,name='mark_attendance'),
    path('get-students-by-course/',views.get_students_by_course,name='get_students_by_course'),
]
```

## views.py

appFaculty/views.py

```
from django.shortcuts import render,redirect
from django.http import HttpResponse,JsonResponse
from .models import Faculty,ActivityLogsFaculty,FacultyAssignedSubject
from appErpAdmin.models import Courses,University,CourseSessions,StudentEnrollment
from django.db.models import Q
from django.contrib.auth import authenticate
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.authentication import JWTAuthentication
from datetime import datetime
from .models import FacultyWiseAttendance
from .models import StudentWiseAttendance


# Create your views here.
def faculty_login(request):

    course_session_it = CourseSessions.objects.filter(

        Q(complete_name__startswith='BCA') |
        Q(complete_name__startswith='BSC') |
        Q(complete_name__startswith='MCA')
    )
    course_session_manag = CourseSessions.objects.filter(
```

```
    Q(complete_name__startswith='BBA') |
    Q(complete_name__startswith='BCOM') |
    Q(complete_name__startswith='MBA') |
    Q(complete_name__startswith='PGDM')
)


if request.method=="POST":
    faculty_id = request.POST.get("faculty_id")
    password = request.POST.get("password")

    try:
        faculty = Faculty.objects.get(faculty_id=faculty_id)

        #temp pwd check

        # jwt code

        user = authenticate(
        username=faculty_id,
        password=password
        )

        # if password == faculty.faculty_id:
        if user is not None:
            # jwt code
            refresh = RefreshToken.for_user(faculty)
            access_token = str(refresh.access_token)
            # print(access_token)
            request.session['access_token'] = access_token

            #

            token = request.session.get('access_token')
            #store into logs of activity
            if token:
                jwt_auth = JWTAuthentication()
                validated_token = jwt_auth.get_validated_token(token)
                ActivityLogsFaculty.objects.create(
                    faculty=faculty,
                    action='Login'
                )
                # print(faculty.faculty_id)
                request.session['f_id']=faculty.faculty_id
                return render(request,
                    'faculty_module/faculty_dashboard.html',{
                        'faculty':faculty,
                        'course_session_it':course_session_it,
                        'course_session_manag':course_session_manag,
                        'fid':faculty.faculty_id,
                    }
                            )
        else:
            return render(request,
                'faculty_module/login.html',{
                    'error':'Invalid Password'
                }
                        )

    except Faculty.DoesNotExist:
        return render(request,
                      'faculty_module/login.html',
                      {
                          "error":'faculty id invalid'
                      }
                      )
```

```
    return render(request, "faculty_module/login.html")


def faculty_logout(request):

    if request.method=="POST":
        faculty_id=request.POST.get('faculty_id')
        try:
            faculty = Faculty.objects.get(
                faculty_id=faculty_id
            )
            #store logout activity

            ActivityLogsFaculty.objects.create(
                faculty=faculty,
                action='Logout'
            )
        except Faculty.DoesNotExist:
            pass

    return redirect('faculty_login')



def makeSchedulesIT(request):

    faculties = Faculty.objects.filter(

        optionselected__in=[
            'opt1_IT_Only',
            'opt3_Both_roles'
        ]

    ).prefetch_related(
        'assigned_subjects'
    )

    context = {

        'faculties': faculties

    }

    return render(

        request,

        'faculty_module/schedulerIT.html',

        context

    )

def makeSchedulesM(request):

    faculties = Faculty.objects.filter(

        optionselected__in=[
            'opt2_Manag_Only',
            'opt3_Both_roles'
        ]

    ).prefetch_related(
```

```
        'assigned_subjects'
    )

    context = {

        'faculties': faculties

    }

    return render(

        request,

        'faculty_module/schedulerManagement.html',

        context

    )

def add_faculty_subject(request):

    if request.method == "POST":

        faculty_id = request.POST.get('faculty_id')

        subject_name = request.POST.get('subject_name')

        subject_code = request.POST.get('subject_code')

        semester = request.POST.get('semester')

        session = request.POST.get('session')

        try:

            faculty = Faculty.objects.get(
                faculty_id=faculty_id
            )

            FacultyAssignedSubject.objects.create(

                faculty=faculty,

                subject_name=subject_name,

                subject_code=subject_code,

                semester=semester,

                session=session

            )

        except Faculty.DoesNotExist:

            pass

    return redirect('makeSchedulesIT')

# --
def mark_attendance(request):

    faculty_id = request.session.get('f_id')
    # faculty = Faculty.objects.get(
```

```
    #     faculty_id='fa010'
    # )
# point of error
    faculty = Faculty.objects.get(
        faculty_id=faculty_id
    )

    assigned_subjects = FacultyAssignedSubject.objects.filter(
        faculty=faculty.faculty_id
    )

    courses = CourseSessions.objects.all()

    if request.method == 'POST':

        subject = request.POST.get('subject')

        course_id = request.POST.get('course')

        present_students = request.POST.get(
            'present_students'
        )

        course = CourseSessions.objects.get(
            id=course_id
        )

        FacultyWiseAttendance.objects.create(

            attendance_date=datetime.now(),

            course_name=course.complete_name,

            faculty_id=faculty.faculty_id,

            faculty_name=faculty.faculty_name,

            subject=subject,

            attendance=present_students
        )

        if present_students:

            present_students_list = present_students.split(',')

            for student_id in present_students_list:

                StudentWiseAttendance.objects.create(

                    attendance_date=datetime.now(),

                    course_name=course.complete_name,

                    subject=subject,

                    faculty_id=faculty.faculty_id,

                    student_id=student_id
                )

    context = {
        'faculty': faculty,
        'subjects': assigned_subjects,
```

```
        'courses': courses,
        'current_datetime': datetime.now()
    }

    return render(
        request,
        'faculty_module/mark_attendance.html',
        context
    )

def get_students_by_course(request):

    course_id = request.GET.get('course_id')

    enrollments = StudentEnrollment.objects.filter(
        course_id=course_id
    )

    student_data = []

    for enrollment in enrollments:

        student = enrollment.student

        student_data.append({
            'id': student.id,
            'student_id': student.student_id,
            'name': student.name
        })

    return JsonResponse(student_data, safe=False)
# --
```

# 4. App: appMainsite

## admin.py

appMainsite/admin.py

```
from django.contrib import admin

# Register your models here.
```

## models.py

appMainsite/models.py

```
from django.db import models

# Create your models here.


class MainsiteEnquiryForm(models.Model):
    name = models.CharField(max_length=100)
    mobile = models.CharField(max_length=13)
    from_place=models.CharField(max_length=20)
    course= models.CharField(max_length=20)
    created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    db_table = "mainsite_enquiry_form"
```

## views.py

appMainsite/views.py

```
from django.shortcuts import render,redirect
from .models import MainsiteEnquiryForm

# Create your views here.
# Create your views here.
def mainsite(request):
    return render(request,"MainSite/mainsite.html")

def mainsite(request):
    if request.method =="POST":
        MainsiteEnquiryForm.objects.create(
            name=request.POST.get("name"),
            mobile=request.POST.get("mobile"),
            from_place=request.POST.get("from_place"),
            course=request.POST.get("course",)
        )
        return redirect("index")

    return render(request,"MainSite/mainsite.html")
```

# 5. App: appStudent

## admin.py

appStudent/admin.py

```
from django.contrib import admin

# Register your models here.
```

## forms.py

appStudent/forms.py

```
from django import forms
from .models import Student

class StudentImageUploadForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['image']
```

## models.py

appStudent/models.py

```
from django.db import models
from appErpAdmin.models import CourseSessions
from django.contrib.auth.models import User

# Create your models here.

class Student(models.Model):
    # jwt code add user field
    user = models.OneToOneField(
    User,
    on_delete=models.CASCADE,
    null=True
    )
    #
    name = models.CharField(max_length=25)
    student_id = models.CharField(max_length=12,unique=True)
    course = models.CharField(max_length=6)
    email = models.EmailField(unique=True)
    session= models.CharField(max_length=10)
    image=models.ImageField(upload_to='students/')
    date_created = models.DateTimeField(auto_now_add=True)


class ActivityLogs(models.Model):

    ACTION_CHOICES = [
        ('Login', 'Login'),
        ('Logout', 'Logout'),
    ]

    student = models.ForeignKey(
        Student,
        on_delete=models.CASCADE
    )

    action = models.CharField(
        max_length=10,
        choices=ACTION_CHOICES
```

```
    )

    timestamp = models.DateTimeField(
        auto_now_add=True
    )

    def __str__(self):
        return f"{self.student.student_id} - {self.action}"
```

## urls.py

appStudent/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_login, name='student_login'),
    path('logout/', views.student_logout, name='student_logout'),
    path('upload-image/',views.upload_student_image,name='upload_student_image'),
    path('attendance-dashboard/',views.student_attendance_dashboard,name='student_attendance_dashboard'),


]
```

## views.py

appStudent/views.py

```
from django.shortcuts import render,redirect,get_object_or_404
from django.http import HttpResponse,JsonResponse
from .models import Student, ActivityLogs
from .forms import StudentImageUploadForm
from django.contrib.auth import authenticate
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.authentication import JWTAuthentication
from appFaculty.models import Faculty, StudentWiseAttendance
from appFaculty.models import FacultyWiseAttendance
from django.db.models import Count

def student_login(request):
    if request.method=="POST":
        student_id = request.POST.get("student_id")
        password = request.POST.get("password")

        try:
            student = Student.objects.get(student_id=student_id)

            #temp pwd check
# ------------------------------------------------------------------
            user = authenticate(
            username=student_id,
            password=password
            )
            # if password == student.student_id:
            if user is not None:
                # jwt code
                refresh = RefreshToken.for_user(student)
                access_token = str(refresh.access_token)
                # print(access_token)
                request.session['access_token'] = access_token
                request.session['s_id']=student_id
                print(student_id)
                #
```

```
                token = request.session.get('access_token')
                if token:
                    jwt_auth = JWTAuthentication()
                    validated_token = jwt_auth.get_validated_token(token)
                    #store into logs of activity
                    ActivityLogs.objects.create(
                        student=student,
                        action='Login'
                    )

                    return render(request,
                        'student_module/student_dashboard.html',{
                            'student':student
                        }
                                )
# ---------------------------------------------------------------------
            else:
                return render(request,
                    'student_module/login.html',{
                        'error':'Invalid Password'
                    }
                            )

        except Student.DoesNotExist:
            return render(request,
                          'student_module/login.html',
                          {
                              "error":'Student id invalid'
                          }
                          )



    return render(request, 'student_module/login.html')

def student_logout(request):

    if request.method=="POST":
        student_id=request.POST.get('student_id')
        try:
            student = Student.objects.get(
                student_id=student_id
            )
            #store logout activity

            ActivityLogs.objects.create(
                student=student,
                action='Logout'
            )
        except Student.DoesNotExist:
            pass

    return redirect('student_login')

def upload_student_image(request):

    if request.method == 'POST':

        student_id = request.POST.get('student_id')

        try:
            student = Student.objects.get(student_id=student_id)

        except Student.DoesNotExist:
            return render(
```

```
                request,
                'student_module/login.html',
                {
                    'error': 'Student id invalid'
                }
            )

        form = StudentImageUploadForm(
            request.POST,
            request.FILES,
            instance=student
        )

        if form.is_valid():
            form.save()

            return render(
                request,
                'student_module/student_dashboard.html',
                {
                    'student': student,
                    'success': 'Image uploaded successfully'
                }
            )

    return redirect('student_login')

    if request.method == 'POST':

        student_id = request.POST.get('student_id')

        try:
            student = Student.objects.get(student_id=student_id)

        except Student.DoesNotExist:
            return render(request,
                'student_module/upload_image.html',
                {
                    'error': 'Student ID invalid'
                }
            )

        form = StudentImageUploadForm(
            request.POST,
            request.FILES,
            instance=student
        )

        if form.is_valid():
            form.save()

            return render(request,
                'student_module/student_dashboard.html',
                {
                    'student': student
                }
            )

    return render(request, 'student_module/upload_image.html')


def student_attendance_dashboard(request):

    student_id = request.session.get(
        's_id'
    )
```

```
attendance_logs = StudentWiseAttendance.objects.filter(
    student_id=student_id
).order_by('-attendance_date')

# faculty_map = {
# f.faculty_id: f.faculty_name
# for f in Faculty.objects.all()
# }
faculty_map = {
f.faculty_id: {
    'name': f.faculty_name,
    'designation': f.faculty_designation
}
for f in Faculty.objects.all()
}

# for log in attendance_logs:
#     log.faculty_name = faculty_map.get(log.faculty_id, "Unknown")

for log in attendance_logs:
    faculty = faculty_map.get(log.faculty_id)

    if faculty:
        log.faculty_display = (
        f"{faculty['name']} ({faculty['designation']})"
        )
    else:
        log.faculty_display = "Unknown"

attendance_summary = []

subjects = StudentWiseAttendance.objects.filter(
    student_id=student_id
).values(
    'subject'
).distinct()

for sub in subjects:

    subject_name = sub['subject']

    total_classes = FacultyWiseAttendance.objects.filter(
        subject=subject_name
    ).count()

    present_classes = StudentWiseAttendance.objects.filter(
        student_id=student_id,
        subject=subject_name
    ).count()

    percentage = 0

    if total_classes > 0:

        percentage = (
            present_classes / total_classes
        ) * 100

    attendance_summary.append({

        'subject': subject_name,

        'total_classes': total_classes,
```

```
        'present_classes': present_classes,

        'percentage': round(percentage, 2)

    })

context = {

'attendance_logs': attendance_logs,

'attendance_summary': attendance_summary,

'student_id': student_id
}

return render(
    request,
    'student_module/student_attendance.html',
    context
)
```

# 6. Templates (sample)

## login_HOD/index.html

templates/login_HOD/index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    {% load static %}
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <link rel="stylesheet" href="{% static 'login_HOD/styles.css'%}">
</head>

<body>
    <div class="top-panel">
        .
    </div>

    <div class="shape rect image-responsive-custom"></div>

    <div class="img-container">
        <img src="{% static 'login_HOD/HOD_LOGIN.png' %}" class="full-img mobile-adjust img-fluid">
    </div>

    <!-- code of the input -->
    <div class="shape rect rectangle-input-container">
    </div>
    <!-- rect: image -->
    <div class="shape rect image-login-logo">
    </div>
    <!-- rect: Rectangle -->
    <div class="shape rect rectangle-seperator">
    </div>
    <!-- text: College id :  -->
    <div class="shape text college-id-cd07efa9893b">
        <div class="text-node-html" id="html-text-node-58cff496-0ba4-8066-8007-cd07efa9893b" data-x="27"
data-y="151">
            <div class="root rich-text root-0" xmlns="http://www.w3.org/1999/xhtml">
                <div class="paragraph-set root-0-paragraph-set-0">
                    <p class="paragraph root-0-paragraph-set-0-paragraph-0" dir="ltr"><span
                            class="text-node root-0-paragraph-set-0-paragraph-0-text-0">College id :
</span>

                </div>
            </div>
        </div>
    </div>
    <input
                            id="collegeInput" type="text" class="custom-input" /></p>
    <!-- code of the input -->

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

<!-- code of modal -->
 <div class="modal fade" id="loginFailModal">
<div class="modal-dialog modal-dialog-centered">
<div class="modal-content">

<div class="modal-header">
<h5 class="modal-title">Login Failed</h5>
</div>
```

```
<div class="modal-body">
Incorrect credentials
</div>

<div class="modal-footer">
<button class="btn btn-secondary" data-bs-dismiss="modal">
Close
</button>
</div>

</div>
</div>
</div>
<!-- code of modal -->

<!-- Modal controller Script -->
 <script>

let step=1

document.querySelector(".image-login-logo")
.addEventListener("click",function(){

let input=document.getElementById("collegeInput")

let label=document.querySelector(
".root-0-paragraph-set-0-paragraph-0-text-0"
)

if(step===1){

label.innerText="Pass :"
input.value=""
input.type="password"

step=2
return
}

if(step===2){

if(input.value==="1234"){

alert("Login success")

}else{

new bootstrap.Modal(
document.getElementById("loginFailModal")
).show()

}

}

})

</script>
<!-- Modal controller Script -->


</body>

</html>
```

## faculty_module/faculty_dashboard.html

templates/faculty_module/faculty_dashboard.html

```
<!doctype html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>College ERP Dashboard</title>

  <!-- FONT AWESOME -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.1/css/all.min.css"
/>

  <style>
    /* =========================
       GLOBAL STYLES
    ========================== */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: Arial, Helvetica, sans-serif;
    }

    body {
      background: #dcdcdc;
      min-height: 100vh;
    }

    a {
      text-decoration: none;
      color: white;
    }

    ul {
      list-style: none;
    }

    /* =========================
       MAIN LAYOUT
    ========================== */
    .container {
      width: 100%;
      min-height: 100vh;
      display: flex;
    }

    /* =========================
       SIDEBAR
    ========================== */
    .sidebar {
      width: 260px;
      background: #b06635;
      color: white;
      transition: 0.3s;
      overflow: hidden;
      position: fixed;
      left: 0;
      top: 0;
      bottom: 0;
      z-index: 1000;
    }

    .sidebar.collapsed {
```

```
  width: 80px;
}

.logo {
  height: 70px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 24px;
  font-weight: bold;
  border-bottom: 1px solid rgba(255, 255, 255, 0.2);
}

.menu {
  padding-top: 20px;
}

.menu li {
  margin: 8px 0;
}

.menu a {
  display: flex;
  align-items: center;
  gap: 15px;
  padding: 15px 20px;
  transition: 0.3s;
  white-space: nowrap;
}

.menu a:hover {
  background: rgba(255, 255, 255, 0.15);
}

.menu i {
  font-size: 20px;
  min-width: 25px;
  text-align: center;
}

.sidebar.collapsed .menu-text {
  display: none;
}

/* =========================
   MAIN CONTENT
========================= */
.main {
  margin-left: 260px;
  width: 100%;
  transition: 0.3s;
  padding: 20px;
}

.main.expanded {
  margin-left: 80px;
}



/* =========================
   TOPBAR
========================= */
.topbar {
  width: 100%;
  height: 70px;
```

```
  background: white;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 20px;
  border-radius: 10px;
  margin-bottom: 20px;
}

.left-top {
  display: flex;
  align-items: center;
  gap: 20px;
}

.menu-btn {
  font-size: 24px;
  cursor: pointer;
  color: #02124a;
}

.student-title {
  font-size: 22px;
  font-weight: bold;
  color: #02124a;
}

.logout-btn button {
  font-size: 26px;
  color: #02124a;
  cursor: pointer;
  border-style: none;

}

/* =========================
   CONTENT AREA
========================= */
.content {
  width: 100%;
  min-height: 500px;
  background: #ececec;
  border-radius: 10px;
  padding: 20px;
  position: relative;
}

/* STUDENT IMAGE */
.student-profile {
  width: 120px;
  height: 120px;
  border-radius: 50%;
  background: #b8c3ea;
  position: absolute;
  right: 20px;
  top: 20px;
}

/* =========================
   PALETTE DIVS
========================= */

/*
  Case Study:
  A -> My Details
  B -> Routine
  C -> Attendance
```

```
  Default:
  A = display block
  B,C = display none
*/

.palette {
  margin-top: 160px;
}

.page {
  display: none;
  padding: 20px;
  background: white;
  border-radius: 10px;
}

.page.active {
  display: block;
}

.page h2 {
  margin-bottom: 15px;
  color: #02124a;
}

.page p {
  line-height: 1.7;
  color: #333;
}

/* =========================
   RESPONSIVE DESIGN
========================= */

/* TABLETS */
@media (max-width: 992px) {
  .sidebar {
    width: 80px;
  }

  .sidebar .menu-text {
    display: none;
  }

  .main {
    margin-left: 80px;
  }

  .main.expanded {
    margin-left: 260px;
  }

  .sidebar.open {
    width: 260px;
  }

  .sidebar.open .menu-text {
    display: inline;
  }
}

/* MOBILE */
@media (max-width: 768px) {
  .sidebar {
    left: -260px;
```

```
        width: 260px;
      }

      .sidebar.open {
        left: 0;
      }

      .main {
        margin-left: 0;
      }

      .main.expanded {
        margin-left: 0;
      }

      .topbar {
        padding: 0 15px;
      }

      .student-title {
        font-size: 18px;
      }

      .content {
        padding: 15px;
      }

      .student-profile {
        width: 90px;
        height: 90px;
      }

      .palette {
        margin-top: 120px;
      }
    }
  </style>
</head>

<body>
  <!-- ========================
       CONTAINER
  ======================== -->
  <div class="container">
    <!-- ========================
         SIDEBAR
    ======================== -->
    <aside class="sidebar" id="sidebar">
      <div class="logo">College ERP</div>

      <ul class="menu">
        <!-- LINK 1 -> DIV A -->
        <li>
          <a href="#" onclick="showPage('pageA')">
            <i class="fa-solid fa-user"></i>
            <span class="menu-text">My Details</span>
          </a>
        </li>
        <li>
          <a href="#" onclick="showPage('pageA1')">
            <i class="fa-solid fa-user"></i>
            <span class="menu-text">Forms Section</span>
          </a>
        </li>

        <!-- LINK 2 -> DIV B -->
        <li>
```

```
          <a href="#" onclick="showPage('pageB')">
            <i class="fa-solid fa-calendar-days"></i>
            <span class="menu-text">My Classes</span>
          </a>
        </li>

        <!-- LINK 3 -> DIV C -->
        <li>
          <a href="#" onclick="showPage('pageC')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Attendance</span>
          </a>
        </li>

        <!-- LINK 4 -> DIV D -->
        <li>
          <a href="#" onclick="showPage('pageD')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Exams Module</span>
          </a>
        </li>

        <!-- LINK 5 -> DIV E -->
        <li>
          <a href="#" onclick="showPage('pageE')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Salary</span>
          </a>
        </li>

        <!-- LINK 6 -> DIV F -->
        <li>
          <a href="#" onclick="showPage('pageF')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Notices</span>
          </a>
        </li>

        <!-- LINK 7 -> DIV G -->
        <li>
          <a href="#" onclick="showPage('pageG')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Contacts</span>
          </a>
        </li>
        <!-- LINK 8 -> DIV H -->
        <li>
          <a href="#" onclick="showPage('pageH')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Account Settings</span>
          </a>
        </li>

        {% if faculty.faculty_designation == 'HOD - IT' or faculty.faculty_designation == 'HOD -
Management' %}
        <li>
          <a href="#" onclick="showPage('pageI')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Issue Notices</span>
          </a>
        </li>
        {% if faculty.faculty_designation == 'HOD - IT'%}
        <li>
          <a href="makeSchedulesIT/" onclick="showPage('pageJ')">
            <i class="fa-solid fa-chart-column"></i>
            <span class="menu-text">Create Time Tables</span>
          </a>
```

```
    </li>
    <li>
      <a href="#" onclick="showPage('pagek')">
        <i class="fa-solid fa-chart-column"></i>
        <span class="menu-text">All IT Batches</span>
      </a>
    </li>
    {% endif %}

    {% if faculty.faculty_designation == 'HOD - Management'%}
    <a href="makeSchedulesM/" onclick="showPage('pageJ')">
      <i class="fa-solid fa-chart-column"></i>
      <span class="menu-text">Create Time Tables</span>
    </a>
    </li>
    <li>
      <a href="#" onclick="showPage('pagek1')">
        <i class="fa-solid fa-chart-column"></i>
        <span class="menu-text">All Management Batches</span>
      </a>
    </li>
    {% endif %}
    {% endif %}

  </ul>
</aside>

<!-- ========================
     MAIN SECTION
========================= -->
<main class="main" id="main">
  <!-- TOPBAR -->
  <div class="topbar">
    <div class="left-top">
      <i class="fa-solid fa-bars menu-btn" id="menuBtn"></i>

      <div class="student-title">Faculty Dashboard</div>
    </div>

    <div class="logout-btn">
      <form method="POST" action="{% url 'faculty_logout'%}">
        {% csrf_token %}

        <input type="hidden" name="faculty_id" value="{{ faculty.faculty_id }}" />
        <button type="submit">Logout</button>
      </form>
    </div>
  </div>

  <!-- CONTENT -->
  <div class="content">
    <!-- PROFILE -->
    <div class="student-profile"></div>

    <!-- PALETTE -->
    <div class="palette">
      <!-- ========================
           DIV A
           DEFAULT DISPLAY BLOCK
      ========================= -->
      <div class="page active" id="pageA">
        <h2>Welcome {{ faculty.faculty_name }}</h2>

        <p>Faculty ID : {{ faculty.faculty_id }}</p>

        <p>Designation : {{ faculty.faculty_designation }}</p>
```

```
            <p>Option Selected : {{ faculty.optionselected }}</p>
          </div>

          <div class="page active" id="pageA1">
            <h2>Forms Section</h2>
              <a target="_blank" href="mark-attendance/" style="font-size: large;color: #0662d2;">Mark
Student Attendance [Form 3]</a>
          </div>

          <!-- ========================
               DIV B
          ========================== -->
          <div class="page" id="pageB">
            <h2>Routine</h2>

            <p>Monday : DBMS</p>

            <p>Tuesday : Operating Systems</p>

            <p>Wednesday : Django Lab</p>
          </div>

          <!-- ========================
               DIV C
          ========================== -->
          <div class="page" id="pageC">
            <h2>Attendance</h2>

            <p>DBMS : 89%</p>

            <p>Operating Systems : 91%</p>

            <p>Django Lab : 95%</p>
          </div>
          <!-- -------------------------HOD PANEL -->
          <div class="page" id="pageI">
            <h2>Issue Notices</h2>
            =
          </div>

          <div class="page" id="pageJ">
            <h2>Create Time Tables</h2>

          </div>

          <div class="page" id="pagek">
            <h2>List of IT batches</h2>
            <table border="1">
              <tr>
                <th>ID</th>
                <th>Course Name</th>
              </tr>

              {% for course in course_session_it %}
              <tr>
                <td>{{ course.id }}</td>
                <td>{{ course.complete_name }}</td>
              </tr>
              {% endfor %}
            </table>
          </div>
          <div class="page" id="pagek1">
            <h2>List of Management Batches</h2>
            <table border="1">
              <tr>
                <th>ID</th>
```

```
              <th>Course Name</th>
            </tr>

            {% for course in course_session_manag %}
            <tr>
              <td>{{ course.id }}</td>
              <td>{{ course.complete_name }}</td>
            </tr>
            {% endfor %}
          </table>
        </div>
        <!-- -------------------------HOD PANEL -->
      </div>
    </div>
  </main>
</div>

<!-- =========================
     JAVASCRIPT
========================== -->
<script>
  /*
=============================================
SIDEBAR TOGGLE
=============================================

Desktop:
Collapse sidebar

Mobile:
Open/close sidebar
*/

  const menuBtn = document.getElementById("menuBtn");
  const sidebar = document.getElementById("sidebar");
  const main = document.getElementById("main");

  menuBtn.addEventListener("click", () => {
    if (window.innerWidth <= 768) {
      sidebar.classList.toggle("open");
    } else {
      sidebar.classList.toggle("collapsed");
      main.classList.toggle("expanded");
    }
  });

  /*
=============================================
PAGE SWITCHING LOGIC
=============================================

A -> pageA
B -> pageB
C -> pageC

Default:
pageA = display block
Others hidden
*/

  function showPage(pageId) {
    // get all pages
    const pages = document.querySelectorAll(".page");

    // hide all pages
    pages.forEach((page) => {
      page.classList.remove("active");
```

```
      });

      // show selected page
      document.getElementById(pageId).classList.add("active");

      /*
      Auto close sidebar on mobile
      after clicking a menu item
    */
      if (window.innerWidth <= 768) {
        sidebar.classList.remove("open");
      }
    }
  </script>
</body>

</html>
```

# Chapter : 18

# References and Bibliography

# College ERP

References and Bibliography

Books

Holovaty, A., & Kaplan-Moss, J. The Definitive Guide to Django: Web Development Done Right. Apress Publications.

William S. Vincent. Django for Professionals: Production Websites with Python & Django. WelcomeToCode.

Beaulieu, A. Learning SQL. O'Reilly Media.

Kleppmann, M. Designing Data-Intensive Applications. O'Reilly Media.

Turnbull, J. The Docker Book: Containerization is the New Virtualization. James Turnbull Publishing.

Richards, M., & Ford, N. Fundamentals of Software Architecture. O'Reilly Media.

Official Documentation and Online References

Django Software Foundation. Django Documentation. Available at:

https://docs.djangoproject.com/

Amazon Web Services (AWS). AWS Official Documentation. Available at:

https://docs.aws.amazon.com/

Docker Inc. Docker Documentation. Available at:

https://docs.docker.com/

PostgreSQL Global Development Group. PostgreSQL Documentation. Available at:

https://www.postgresql.org/docs/

Python Software Foundation. Python Documentation. Available at:

https://docs.python.org/

Bootstrap Team. Bootstrap Documentation. Available at:

https://getbootstrap.com/docs/

MDN Web Docs. HTML, CSS and JavaScript Documentation. Available at:

https://developer.mozilla.org/

GitHub Inc. GitHub Documentation. Available at:

https://docs.github.com/

Bibliography

The development of the College ERP System was carried out using Django as the primary web framework, PostgreSQL as the database management system, Docker for containerization, and Amazon Web Services (AWS) EC2 for cloud deployment. Various official documentation resources, technical books, and online references were consulted throughout the project lifecycle for system design, implementation, deployment, database management, and maintenance. The Django, PostgreSQL, Docker, and AWS official documentation served as the primary sources for understanding framework architecture, container orchestration, database administration, and cloud infrastructure deployment. Additional references from software architecture and database design literature were utilized to ensure the system followed industry-standard development practices and scalable design principles.